Poznaj oprogramowanie ERP,
które pracuje na Twój sukces
przez 365 dni w roku!

**Systemowy rozwój
Twojej firmy.**

API - wydruki

Spis treści

API - wydruki 3
- ReportResult 4
- Klasy związane z wydrukami 13
- XtraReport 16

API - wydruki

Sekcja opisuje API Soneta związane z wydrukami. API publiczne, przeznaczone do wykorzystywania w dodatkach to ReportResult. Pozostałe klasy, nawet jeśli są publiczne i dostępne w bibliotekach Soneta, w ogólności nie są przeznaczone do użytku zewnętrznego. Ich kształt, zależności i implementacja zmienia się stosunkowo rzadko, jednak ew. zmiany są zazwyczaj niezgodne wstecz.

ReportResult

W programie jest wiele mechanizmów tworzenia wydruków. Zależą od rodzaju wydruku (ASPX / REPX / DOTX), wersji oprogramowania (okienkowa vs webowa), konstrukcji wydruku (z parametrami lub bez, na podstawie listy lub formularza), implementacji funkcji specjalnych w skryptach i innych czynników. API `ReportResult` powstało w celu schowania całej złożoności wydruków w implementacji wewnętrznej, a wystawienia do użytku zewnętrznego podstawowych funkcji wydruków, wspólnych dla wszystkich ich rodzajów. `ReportResult` (namespace `Soneta.Business.UI`) pozwala na:

- utworzenie dokumentu PDF lub HTML na podstawie podanego wzorca wydruku (pliku .aspx, .repx lub .dotx) i innych parametrów (tryb niezależny od interfejsu)
- wywołanie wydruku o określonej nazwie, czyli zasymulowanie kliknięcia w interfejsie na konkretny wydruk (tryb interaktywny, z użyciem interfejsu)
- wywołanie wydruku o określonym wzorcu lub typie, z możliwością wykonania dowolnej procedury na wyniku wydruku

Istnieje kilka scenariuszy wykorzystania `ReportResult`. Od strony programistycznej w API `ReportResult` można wydzielić dwa zasadniczo różne tryby:

- wykorzystanie własnej instancji `ReportResult` wraz z serwisem `IReportService`
- wykorzystanie własnej instancji `ReportResult` jako rezultatu operacji

ReportResult z IReportService

`IReportService` to prosty interfejs, umożliwiający wydruk wydruku do strumienia danych (PDF), do string (HTML) lub bezpośrednio na drukarkę (wydruk serwerowy). Interfejs posiada trzy metody:

```
public interface IReportService {
    Stream GenerateReport(ReportResult reportResult);
    string GenerateReportStr(ReportResult reportResult);
    void PrintReport(ReportResult reportResult, bool archive = false, string archivePath = "");
}
```

Serwis wydruku pobiera się z sesji, jak większość serwisów służących do przetwarzania danych sesyjnych:

```
IReportService service = Session.GetRequiredService<IReportService>();
```

Więcej o ogólnej konstrukcji serwisów tutaj.

Instancja obiektu `ReportResult` zawiera elementy potrzebne do utworzenia wydruku. W najprostszym ujęciu jest to źródło danych (`Context`), wzorzec wydruku i konfiguracja:

```
ReportResult rr = new ReportResult()
{
    Context = context,
    Format = ReportResultFormat.PDF,
```

```
    DataType = typeof(DokumentHandlowy),
    TemplateFileName = "faktura.repx"
};
```

Tak przygotowany obiekt `ReportResult` utworzy wydruk PDF z obiektu typu `DokumentHandlowy`, z wzorca `faktura.repx`, na podstawie danych obecnych w przekazanym obiekcie `Context`. Cała procedura związana z wykonaniem wydruku zaimplementowana jest w obiekcie serwisu i metodzie `GenerateReport`:

```
using (Stream stream = service.GenerateReport(rr)) {
       /* zapisz stream do pliku PDF */
}
```

Metoda zwraca strumień zawierający kompletny dokument PDF, gotowy do zapisu do dowolnego pliku. Przy wydruku do HTML przydatna może być alternatywna metoda serwisu, zwracająca string -- `GenerateReportStr`.

ReportResult jako rezultat operacji

Rezultat operacji `ReportResult` można wykorzystać dwojako:

- ze wskazaniem raportu po nazwie - instancja obiektu zawiera wtedy pole `ReportName`
- ze wskazaniem raportu wg wzorca - instancja obiektu zawiera wtedy pola `TemplateFileName` lub `TemplateType`

Ze względu na zmianę reprezentacji wzorców wydruków property TemplateType utrzymywane jest wyłącznie dla zachowania zgodności wstecz i zadziała tylko dla tych wzorców, które przechowywane są nadal w postaci klasy C#.

Dla mechanizmu obsługi `ReportResult` obecność lub brak własności `ReportName`, `TemplateFileName` i/lub `TemplateType` jest kluczowa. W razie wątpliwości można skorzystać z wyliczanej własności `UseReportMenu` - zwraca `true` wtedy, gdy rezultat będzie działał z trybie interaktywnym, tj. z udziałem interfejsu (czyli kiedy nie wskazujemy wzorca lub typu wydruku, tylko jawnie podajemy nazwę).

Właściwość `TemplateFileName` obsługuje również wskazanie pliku XML z ustawieniami raportu, znajdującego się w ścieżce **Narzędzia -> Opcje -> Systemowe -> Pliki** w folderze **Reports** i odpowiednim podfolderze. Zostanie wtedy wykorzystany wzorzec wskazany w danym pliku XML. W przypadku gdy istnieje wiele kontekstów zapisu danego pliku, zostanie wczytana wersja najbardziej odpowiadająca aktualnemu loginowi.

ReportResult ze wskazaniem raportu po nazwie

Jeśli obiekt typu `ReportResult` zwracany jest z workera, obsługiwany jest przez mechanizm rezultatów operacji. W tym przypadku nie tworzymy jawnie serwisu, nie otrzymujemy też gotowego strumienia do zapisu do pliku. Jeśli akcja workera zwraca obiekt typu `ReportResult`, to uruchomienie workera będzie miało ten sam skutek, co uruchomienie wydruku o określonej nazwie z menu wydruków danej listy.

```
    [Action("RR/repx handler", Mode = ActionMode.SingleSession, Target = ActionTarget.Menu |
ActionTarget.ToolbarWithText)]
```

```
    object RR(Context context) {
        return new ReportResult {
            Context = context,
            Format = ReportResultFormat.PDF,
            ReportName = "Dokument/Zwrot podatku dla podróżnych (Tax Free)"
        };
    }
```

Powyższy kod spowoduje otwarcie okna wydruku. Stanie się tak pod warunkiem, że worker zostanie uruchomiony na liście, która zawiera wydruk o takiej nazwie. API pozwala także wskazać widoki, w których framework ma szukać raportu wskazanego przez nazwę. Pozwala także wskazać `DataType`, czyli typ, dla którego raport został zarejestrowany. Poniższa akcja workera wywoła podgląd raportu **cennik** z listy towarów:

```
object RR(Context context) {
    return new ReportResult {
         Context = context,
         DataType = Type.GetType("Soneta.Towary.Towary,Soneta.Handel"),
         ViewNames = new[] { "CennikTowarow" },
         ReportName = "Cennik",
         Preview = true
    }
}
```

W przypadku pustej wartości `ReportName`, zostanie uruchomiony 1-szy raport z listy, co przypomina domyślną akcję wywoływaną przez aplikację w momencie kliknięcia ikony drukarki na dowolnej liście.

Jeśli `ReportResult` użyty jest ze wskazaniem raportu po nazwie, wydruk z parametrami zachowa się tak samo, jakby ten wydruk uruchomić przez standardowy przycisk drukarki z interfejsu użytkownika. Oznacza to, że przed wykonaniem wydruku zostanie otwarte okno parametrów wydruku. Instancja klasy parametrów wypełnionymi na podstawie okna, zostanie dodana do obiektu `Context`, na podstawie którego będzie tworzony wydruk.

ReportResult ze wskazaniem wzorca raportu

W tym trybie `ReportResult` działa jako rezultat operacji, tak samo jak w przypadku wskazania nazwy raportu we właściwości `ReportName`. Jednak nie zostaje w tym przypadku wyświetlone okno wydruku. Wydruk jest tworzony automatycznie (poprzez niejawne wykorzystanie serwisu `IReportService`), a wynik wydruku (PDF) jest wyświetlany użytkownikowi uruchamiającemu worker.

Zamiast nazwy raportu, w instancji obiektu `ReportResult` wskazujemy w tym przypadku wzorzec wydruku, który ma zostać użyty. Property przechowujące plik wzorca to `TemplateFileName`. Może to być plik .aspx lub .repx. Plik oznacza tutaj konfigurację w bazie danych -- *Ustawienia -> Systemowe -> Pliki*. Mechanizm odpowiedzialny za pobieranie konfiguracji z bazy (klasa `ConfigStorageProvider`) próbuje najpierw odnaleźć określoną konfigurację wg ścieżki w bazie, a jeśli to się nie uda, sprawdza analogiczną ścieżkę na dysku.

```
return new ReportResult {
    Context = context,
    Format = ReportResultFormat.PDF,
```

```
    TemplateFileName = "XtraReports/Wzorce użytkownika/cennik.repx",
}
```

W przypadku wydruków aspx, poza `TemplateFileName` można dodatkowo wskazać `TemplateFileSource`. Jest to property przyjmujące wartości `AspxSource.Local` lub `AspxSource.Storage`. `Local` oznacza wyszukiwanie wzorców na dysku, `Storage` oznacza wyszukiwanie wzorców w bazie danych. W przypadku wydruków repx property `TemplateFileSource` jest ignorowane - wzorce wyszukiwane są najpierw w bazie danych, przy czym dzięki opisanej wyżej klasie `ConfigStorageProvider`, możliwe jest pobieranie wzorców z dysku.

Alternatywnie, zamiast ścieżki do wzorca, w przypadku wydruków repx można podać zarejestrowany typ wydruku:

```
return new ReportResult {
    Context = context,
    Format = ReportResultFormat.PDF,
    TemplateType = "Raporty.Cennik, Soneta.Handel.Reports",
}
```

Property TemplateType jest _obsolete_, bo adekwatne jest tylko dla przypadku, w którym wydruk ma postać klasy (raport w postaci kodu C#).

Jeśli `ReportResult` jest użyty z podaniem wzorca wydruku, szczególne znaczenie zaczyna odgrywać property `DataType`. `DataType` określa jakie dane będą wyciągane z kontekstu na potrzeby wydruku.

1. jeśli `DataType` to typ wiersza (np. `typeof(Towar)`), na potrzeby wykonania wydruku w kontekście będzie wyszukiwany pojedynczy obiekt `Row`
2. jeśli `DataType` to typ tabeli (np. `typeof(Towary)`), na potrzeby wykonania wydruku z kontekstu będzie pobierany obiekt `View`, a z widoku zostaną pobrane wszystkie wiersze dostarczane przez ten widok
3. jeśli `DataType` to typ array (np. `typeof(Towar[])`), na potrzeby wykonania wydruku z kontekstu zostanie pobrany typ `Towar[]`, czyli zaznaczone wiersze na aktualnie otwartej liście
4. jeśli `DataType` nie jest podane, czyli ma wartość `null`, zachowanie zależy od miejsca uruchomienia workera: ** jeśli worker uruchomiony jest z formularza, mechanizm zachowuje się jak w przypadku 1 ** jeśli worker uruchomiony jest z listy, mechanizm zachowuje się jak w przypadku 2

Wydruki z parametrami

Wydruk może wymagać dodatkowych parametrów, od których często zależy zarówno warstwa logiczna jak i wizualna raportu. W typowym scenariuszu użycia (kliknięcie na dany wydruk z listy) okno parametrów jest wyświetlane przed wykonaniem wydruku. W użyciu z `ReportResult` zachowanie parametrów zależy od trybu wykorzystania API:

- w trybie rezultatu operacji (tryb interaktywny) ze wskazaniem raportu po nazwie (tryb interaktywny) okno parametrów zostanie wyświetlone użytkownikowi
- w trybie rezultatu operacji (tryb interaktywny) z wykorzystaniem wzorca wydruku jest podobnie - okno parametrów zostanie wyświetlone użytkownikowi jeśli instancji klasy parametrów nie ma w `Context` (oznacza to, że instancję klasy parametrów można podać w `Context`, jeśli wyświetlenie okna nie jest pożądane)
- w trybie automatycznym (opisany wyżej wydruk przez `IReportService`) **okno parametrów nie zostanie wyświetlone**, parametry muszą być przekazane w `Context` lub uzupełnione

za pomocą własnej metody przypisanej do `ReportResult.ParametersHandler`

W trybie interaktywnym, instancja klasy parametrów tworzona jest automatycznie na podstawie okno wyboru parametrów (podobnie do parametrów workerów).

W trybie automatycznym instancję odpowiedniej klasy parametrów należy utworzyć samodzielnie i przekazać do kontekstu w tym samym miejscu, gdzie tworzona jest instancja `ReportResult`, np.:

```
object RR(Context context) {
    var parameters = new KlasaBiznesowa.KlasaParametrowWydruku() { ... };
    context.Set(parameters);
    return new ReportResult {
         Context = context,
         ...
    }
}
```

Możliwe jest też wywołanie okna parametrów wydruku w trybie automatycznym jako okna parametrów workera. Nie ma potrzeby duplikowania klasy parametrów wydruku. Wystarczy zdefiniowanie w workerze kontekstowego property o typie parametrów wydruku:

```
[Context]
public KlasaParametrowWydruku Parms {get; set;}
```

co zapewni nam otwacie okna parametrów workera (wydruku) przed rozpoczęciem wykonywania metody Action workera.

Następnie, jeśli w wydruku korzystamy z innej instancji kontekstu niż kontekst workera, przed wykonaniem metody `GenerateReport` serwisu raportów należy umieścić obiekt tych parametrów w kontekście wydruku:

```
rr.Context.Set(Parms);
```

Zatem jeśli raport korzysta z parametrów, w momencie wykonywania wydruku instancja klasy parametrów musi być obecna w kontekście. Może się tam znaleźć albo poprzez podniesienie i wypełnienie okna parametrów, albo poprzez utworzenie instancji klasy parametrów i przekazanie jej do kontekstu w kodzie workera.

Istnieją dwa zwyczaje dotyczące miejsca, w którym definiowana jest klasa parametrów.

- klasa parametrów w dowolnej przestrzeni nazw, zdefiniowana w warstwie biznesowej, np. `Soneta.Module.Reports.MyReportParameters`
- klasa parametrów zagnieżdżona w klasie samego raportu, np. `MyReports.MyReport.MyReportParams`

Zalecany jest pierwszy sposób. Sposób drugi daje możliwość implementacji wydruku i parametrów w jednym miejscu, ale powoduje trudności w dostępie do parametrów podczas jego wykonywania. Przykład konstrukcji raportu `XtraReports` z zagnieżdżoną definicją klasy parametrów:

```
namespace MyReports{
    /* usings */
    public class MyReport : DevExpress.XtraReports.UI.XtraReport {

      public class MyReportParameters : ContextBase {
         /* constructor */
         MyReportParameters(Context context) : base(context) { ... }

         /* properties */
         ...
      }

        [Context]
        MyReports.MyReport.MyReportParameters Parameters { get; set; }
        /* logika raportu bazująca na Parameters */
    }
}
```

Powyższy przykład dotyczy sytuacji, w której wzorzec zapisany jest w postaci kodu C#. Ten sam schemat jest jednak adekwatny do sytuacji, w której klasa parametrów zagnieżdżona jest w snippecie:

```
namespace MyReports {
    /* usings */
    public class MyReportSnippet : ReportSnippet {     public class MyReportParameters :
ContextBase {

            /* constructor */
            MyReportParameters(Context context) : base(context) { ... }

            /* properties */
            ...
        }

        [Context]
        MyReports.MyReport.MyReportParameters Parameters { get; set; }

        /* logika raportu bazująca na Parameters */
    }
}
```

Ponieważ property `Parameters` oznaczone jest atrybutem `Context`, framework będzie próbował ustawić to property wyszukując odpowiedni typ obiektu w kontekście (analogicznie do

properties-ów w workerach i ekstenderach). Istotne jest w tym przypadku, by wskazać **pełną nazwę typu łącznie z przestrzenią nazw**, a nie tylko samą nazwę klasy.

> **Uwaga!**
>
> Klasa parametrów powinna być wykorzystana wyłącznie do przekazania do raportu wymaganych warunków i opcji sterujących jego zachowaniem. Nie powinna być używana do innych celów - jak np. przygotowanie źródła danych dla raportu. W tym celu należy użyć własnego BusinessDataSource o typie Empty i wypełnić go odpowiednimi danymi w skrypcie raportu, jak opisano np. w tym artykule.

Na zachowanie okna parametrów może także wpływać sposób oznaczenia property parametrów w samym raporcie. W najczęstszym przypadku używany jest atrybut `[Context]` co oznacza, że parametry są wymagane w kontekście w momencie wywołania wydruku:

```
[Context]
Namespace.ParametersClass.Parameters Params { get; set; }
```

Jeśli nie chcemy, aby parametry były bezwzględnie wymagane, można to zmienić za pomocą przełącznika `CreateOnly`

```
[Context(CreateOnly = true)]
Namespace.ParametersClass.Parameters Params { get; set; }
```

Wtedy instancja klasy parametrów zostanie utworzona automatycznie i będzie zawierała domyślne wartości.

Od wersji 2506 ułatwieniem w procesie tworzenia parametrów wydruku w kodzie, jest możliwość przypisania metody `ReportResult.ParametersHandler`. Metoda jest wywoływana w momencie, gdy proces drukowania potrzebuje uzupełnić kontekst o parametry wydruku. Metoda może zostać wywołana wielokrotnie, jeżeli w kontekście brakuje więcej niż jednego typu danych. W parametrach metody otrzymujemy typ którego brakuje, oraz kontekst w którym brakuje instancji obiektu danego typu:

```
ReportResult rr = new ReportResult()
{
    Context = context,
    Format = ReportResultFormat.PDF,
    DataType = typeof(DokumentHandlowy),
    TemplateFileName = "sprzedaz.repx",
    ParametersHandler = (type, context) => {
        dynamic pars = Activator.CreateInstance(type, context)!;
        pars.WgCen = true;
        context.Set(pars);
    }
};
```

Ponieważ typ jest nam znany dopiero w runtime, zamiast wyciągać i ustawiać property za pomocą refleksji, możemy skrócić kod używając słowa kluczowego

```
dynamic.
```

Oczywiście przy takiej konstrukcji w przypadku pomyłki w nazwie property otrzymamy błąd w runtime, a nie w trakcie kompilacji.

Ta metoda jest szybsza i unikamy w ten sposób dołączania bibliotek Soneta.*.Reports do projektu, w przypadku gdy chcemy uzupełnić parametry standardowego raportu.

ReportResult z OutputHandler

Obiekt `ReportResult` można wyposażyć w dodatkową procedurę wykonywaną w momencie, gdy aplikacja wykona wydruk i ma dostęp do gotowego strumienia danych wygenerowanego PDF-a. Jeśli `ReportResult` posiada zdefiniowaną funkcję `OutputHandler`, funkcja wykonywana jest na strumieniu gotowego dokumentu PDF, natomiast rezultat (dowolny `object`) traktowany jest jako docelowy rezultat operacji workera. Inaczej mówiąc, worker zwracający `ReportResult` z podanym `OutputHandler`-em zachowa się tak, jakby worker zwracał obiekt rezultat `OutputHandler`-a (np. obiekt `Row` źródłowego obiektu albo utworzonego załącznika). Sygnatura `ReportResult.OutputHandler` wygląda tak:

```
public Func<Stream, object> OutputHandler;
```

Przykład:

```
    object RR(Context context) {
        return new ReportResult() {
            ...
            OutputHandler = (stream) => GenerateFile(stream, "file.pdf")
        }
    }

    private string GenerateFile(Stream stream, string fileName) {
        var path = Path.Combine(OutputDirectory, fileName);
        using (var fh = File.Create(path)) {
            stream.Seek(0L, SeekOrigin.Begin);
            /* kopiowanie stream do uchwytu pliku */
            fh.Flush();
        }
        if (!File.Exists(path))
            throw new Exception("IO ERROR: Nie udało się zapisać pliku");
        return path;
    }
```

Uwaga!

Użycie funkcji `OutputHandler` ograniczone jest do tego scenariusza, gdy `ReportResult` wykorzystywany jest jako **rezultat operacji, z wydrukiem uruchamianym wg podanego wzorca lub typu** (nie poprzez nazwę wydruku i nie poprzez serwis).

Dodatkowe właściwości

Dodatkowe właściwości `ReportResult`

Szyfrowanie (zabezpieczenie hasłem)

```
public string Encrypt { get; set; }
```

Podanie wartości `Encrypt` powoduje zaszyfrowanie wyjściowego dokumentu PDF, czyli zabezpieczenie go podanym hasłem. Property ma zastosowanie w przypadku użycia `ReportResult` w trybie automatycznym. Natomiast w trybie z użyciem interfejsu do ew. zabezpieczenia hasłem służą dedykowane opcje w UI.

Podpisywanie (certyfikat)

```
public bool VisibleSignature { get; set; } // sygnatura podpisu kwalifikowanego tylko dla
raportów aspx,
public bool Sign { get; set; }             // wywołanie okna z podpisem kwalifikowanym
```

Umożliwia podpisanie dokumentu certyfikatem. Ze względu na konieczność wyświetlenia dodatkowego okna z wyborem certyfikatu, funkcjonalność ograniczona jest do przypadku użycia `ReportResult` w trybie interaktywnym w wersji okienkowej.

Progres

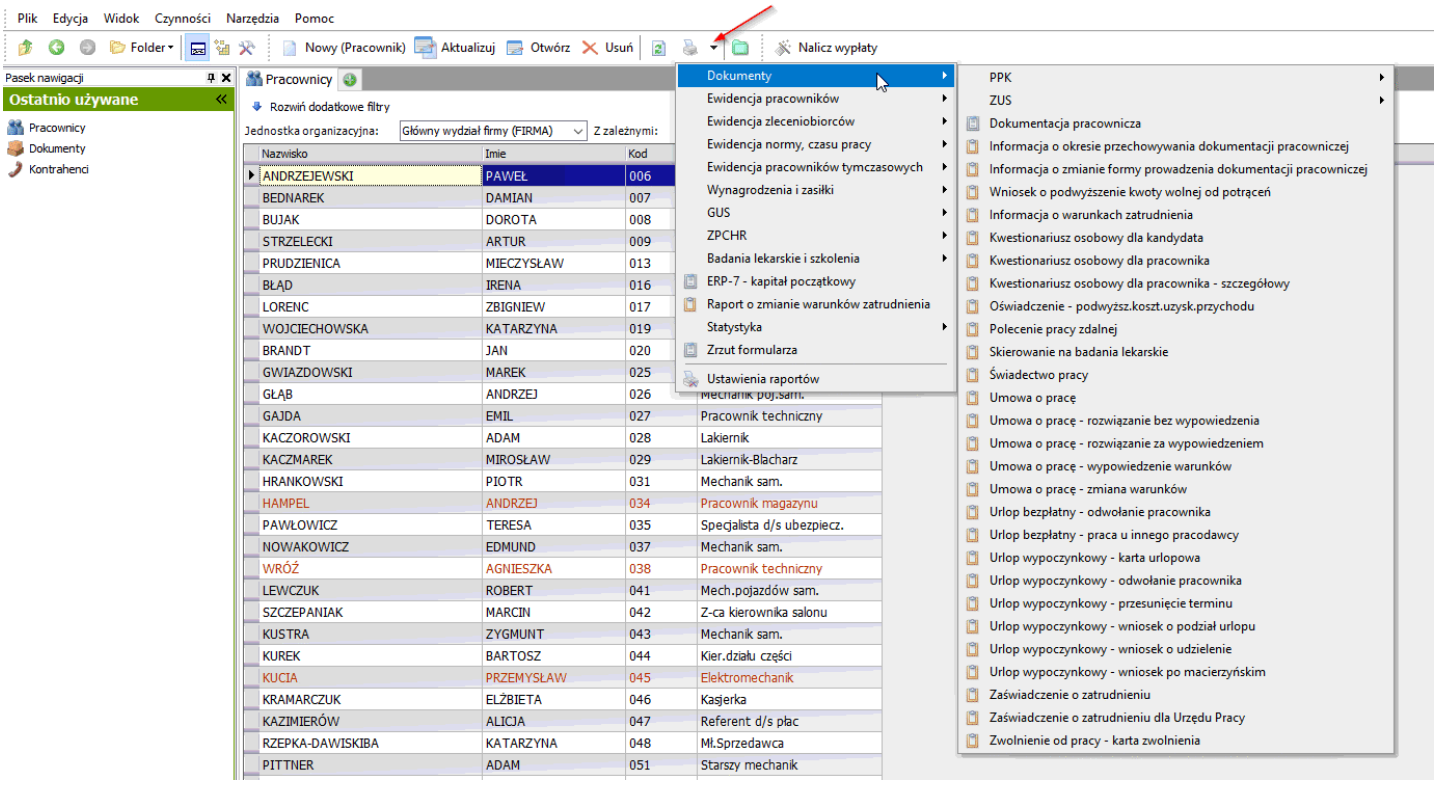
```
public bool SilentProgress { get; set; }
```

Umożliwia schowanie okna paska postępu, które w wersji okienkowej wyświetlane jest automatycznie przy dłużej trwających operacjach.

Klasy związane z wydrukami

Przegląd klas związanych z raportowaniem w aplikacji.

Dla użytkownika systemu, raport czy wydruk jest tożsamy z elementem menu rozwijanego przy ikonie drukarki.



Menu wydruków zaimplementowane jest w klasie o nazwie `ReportsMenu`.

ReportsMenu

Klasa posiada osobną implementację dla wersji okienkowej i webowej:

- `Soneta.Business.Forms.Reports.ReportsMenu` - wersja okienkowa
- `Soneta.Net.Business.Reports.ReportsMenu` - wersja webowa

Obiekt menu to element interfejsu odpowiedzialny za prezentację danych. Menu posiada powiązanie z obiektem managera wydruków.

ReportsManager

Za logikę dotyczącą przygotowania odpowiedniej kolekcji wydruków dla danej listy czy formularza, odpowiedzialny jest `ReportsManager` (namespace `Soneta.Business.UI.Reports`). Obiekty UI korzystają z managera do wyciągnięcia odpowiedniej listy wydruków z assemblies (wydruki standardowe) i z konfiguracji systemu (wydruki użytkownika). Dla każdej listy czy formularza, tworzona jest osobna instancja managera. Jest to wspólny komponent dla wersji okienkowej i HTML.

ReportRef

Kolekcja wydruków obiektu `ReportsManager` przechowywana jest w property `manager.Generators`:

```
List<ReportRef> Generators
```

Każdy wydruk w kolekcji danej listy czy formularza, reprezentowany jest obiektem `ReportRef` (namespace `Soneta.Business.UI.Reports`). Kolekcja obiektów `ReportRef` tworzona jest dynamicznie przy wejściu w menu wydruków danej listy czy formularza.

Obiekt `ReportRef` można traktować jako *referencję / uchwyt do raportu*. Sama referencja nie posiada bezpośredniego odzwierciedlenia w bazie danych. Może natomiast posiadać odpowiednik w postaci pliku XML (property `ReportRef.FileName`), będącego zapisem konfiguracji raportu. Pliki takie tworzone są dla raportów użytkownika (niestandardowych) i zapisywane w w opcjach systemu, w sekcji **Pliki** *(CTRL+F9, CTRL+F)*, w gałęzi **Reports**.

`ReportRef` to klasa abstrakcyjna, posiadająca trzy wcielenia:

- `ReportRef.Standard` - wydruki standardowe, wkompilowane w assemblies
- `ReportRef.Custom` - wydruki niestandardowe, dodawane przez użytkowników i przechowywane w bazie danych w postaci plików konfiguracyjnych XML (ścieżka do pliku dostępna przez property `ReportRef.FileName`)
- `ReportRef.FromForm` - szczególny rodzaj wydruku będącego zapisem okna, czyli *Zrzut formularza*

Instancje `ReportRef` posiadają dowiązania do managera wydruków (`ReportsManager`) oraz generatora (`ReportsGenerator`).

ReportGenerator

Generator to **definicja raportu**. Klasa `ReportGenerator` to klasa abstrakcyjna, posiadająca kilka klas dziedziczących:

- `DxReportGenereator` - generator raportów REPX, namespace `Soneta.Business.UI.DxReports`
- `AspReportGenerator` - generator raportów ASPX, namespace `Soneta.Pritner`
- `FormGenerator` - generator do wydruków formsowych formularzy, namespace `Soneta.Business.Forms.Reports`

Generator dla raportów ASPX dawniej nazywał się `AspGenerator`. W celu ujednolicenia API wydruków dla wersji okienkowej i webowej, od wersji 2010 nazwa klasy została zmieniona. Do wersji 2406 dla zachowania kompatybilności ze starszymi dodatkami, została zachowana klasa `Soneta.Printer.AspGenerator`, oznaczona obecnie jako *obsolete*:

- dla wersji okienkowej, klasa `AspGenerator` została oparta o `WfReportGenerator` (wymaga `Soneta.Forms`):

  ```
  public abstract class WfReportGenerator : ReportGenerator
  public class AspGenerator : WfReportGenerator
  ```

- dla wersji webowej klasa `AspGenerator` jest aliasem do `AspReportGenerator` (dziedziczy wszystkie własności, nie implementuje własnych):

  ```
  public class AspReportGenerator : ReportGenerator
  ```

```
public class AspGenerator : AspReportGenerator
```

Należy mieć na uwadze, że od wersji 2410 metoda **AspGenerator** nie jest już dostępna.

Mnogość klas reprezentujących generatory wynika z zaszłości historycznych i dbałości o zgodność wstecz. W praktyce programista będzie miał do czynienia z raptem dwoma klasami: `DxReportGenereator` oraz `AspReportGenerator`. Obie reprezentują definicję wydruku, zarówno dla okien i wersji html. Samym procesem wydruku zajmuje się kreator.

ReportCreator

Kreator to obiekt odpowiedzialny za wykonanie wydruku - utworzenie w pamięci dokumentu, jego eksport do pliku, ew. wysłanie do drukarki, wyświetlenie podglądu itd. `ReportCreator` to klasa abstrakcyjna, posiadająca kilka wcieleń:

Dla wersji okienkowej:

- `DxReportCreator`
- `AspReportCreator`
- `WfReportCreator`

Dla wersji webowej:

- `AspCreator`
- `DxCreator`

Podsumowanie

Mimo odmiennej implementacji niektórych klas do różnych zastosować, warto zauważyć, że większość klas związanych z wydrukami, a tym samym ogólna logika wydruków w systemie, jest spójna dla wszystkich wersji interfejsu:

- `ReportsMenu` - obiekt UI odpowiedzialny za prezentację kolekcji wydruków
- `ReportsManager` - obiekt odpowiedzialny za przygotowanie odpowiedniej kolekcji wydruków
- `ReportRef` - obiekt reprezentujący raport w menu
- `ReportGenerator` - obiekt reprezentujący definicję wydruku
- `ReportCreator` - obiekt odpowiedzialny za wykonanie wydruku

Wydruki XtraReport

Informacje dotyczące operacyjnej pracy z wydrukami REPX dostępne są w dokumentacji wdrożeniowej. W tym artykułe opisane zostały techniczne i programistyczne aspekty wydruków REPX.

- Formaty zapisu wydruków REPX
  - Korzyści z konwersji
  - Kompatybilność
- Kod logiki raportu
- Konstrukcja wzorców do wersji 2212
  - Format repx
  - Format repx.cs
  - Raporty w bazie
- Konstrukcja wzorców od wersji 2304
  - Format repx
  - Format repx.cs
  - Raporty w bazie
- Korzystanie ze snippet-ów
  - Dziedziczenie
  - Parametry
  - Klasy związane ze snippet-ami
- Konwersja manualna
- Konwersja automatyczna
- Zmiany w DevExpress 23 / enova 2406

Formaty zapisu wydruków REPX

Wzorce wydruków REPX mogą być zapisywane w dwóch formach:

- **C#**
- **XML**

Te dwie formy zapisu wzorców są równoważne funkcjonalnie, różnią się jedynie w detalach implementacyjnych. DevExpress - dostawca platformy raportowania `XtraReports` - za postać domyślną przyjmuje obecnie XML, natomiast postać C# utrzymywana jest dla zapewnienia zgodności wstecz.

Standardowe wydruki REPX dostarczane z systemem enova365 mają postać:

- **C#** - do wersji 2212 włącznie
- **XML** - od wersji 2304

Konieczność zmiany formatu zapisu wydruków REPX została wymuszona zmianami technologicznymi w świecie .NET. Zapis i odczyt wzorców wydruków w formie kodu C# bazował na standardowej funkcjonalności środowiska .NET Framework, czyli na tzw. CodeDOM (DOM - `Document Object Model`). Funkcjonalność ta dostępna była w .NET Framework i umożliwiała posługiwanie się kodem C# tak, jak innymi formatami strukturalnymi. W szczególności, dowolny obiekt (tutaj obiekt wydruku) mógł być automatycznie zapisany (zserializowany) do kodu C# i z takiego kodu wczytany(zdeserializowany, zainstancjonowany). Odpowiadały za to mechanizmy dostępne od ręki w samym środowisku .NET Framework, czyli dostępne w przestrzeniach nazw `System`.

Cały obszar funkcjonalny związany z `CodeDOM` został całkowicie wycofany w środowisku .NET Core. W .NET nie ma obecnie możliwości ani wczytania obiektu z kodu klasy C#, ani zapisania obiektu z powrotem do kodu (nie mówimy tu o kompilacji i dekompilacji, tylko o serializacji obiektów).

Istniejące wzorce w formie C# można dynamicznie skompilować do postaci *assembly*. Mając *assembly* można z kolei pobrać typ skompilowanej klasy i utworzyć przez refleksję instancję obiektu. Uzyskujemy w ten sposób obiekt wydruku `XtraReport` identyczny z tym, jaki powstawał z deserializacji kodu w mechanizmie `CodeDOM`. Dla zachowania zgodności to właśnie rozwiązanie stosuje enova365 począwszy od wersji 2304. Uzyskujemy w ten sposób spójność działania dla wersji 2304 jeszcze działającej na .NET Framework, oraz dla gałęzi produktów działających w .NET. Więcej o samym procesie transformacji z .NET Framework Core w artykule poświęconym .NET6.

Tam gdzie występuje potrzeba wczytania wzorca zapisanego dawniej w formie C#, uruchamiany jest kompilator, analogicznie do kompilacji algorytmów biznesowych w różnych modułach. A z powstałego z kompilacji *assembly* tworzona jest instancja wydruku. W ten sposób w wersjach enova365 dla .NET została zachowana kompatybilność dla wydruków w poprzedniej formie. Jednak zapewnienie analogicznej kompatybilności dla edycji i zapisu wzorców nie jest możliwe, bo nie istnieje alternatywny mechanizm zapisu obiektów do kodu C# w .NET.

Oznacza to, że przejście z wydruków przechowywanych w postaci C# na XML jest konieczne. Dotyczy to nie tylko wydruków standardowych enova365, ale także wszystkich wydruków REPX w instalacjach partnerskich

- zarejestrowanych w dodatkach, zapisanych bazie i w postaci wzorców na dysku. Konwersja z formatu C# na XML jest technicznie bardzo prosta - wystarczy otworzyć wzorzec do edycji i zapisać w nowej postaci. Dostępne są także przykłady pokazujące jak można zautomatyzować konwersję dla wielu wzorców. Aby umożliwić sieci partnerskiej stopniową konwersję swoich wydruków, mechanizmy konieczne do obsługi formatu XML dostępne są już od wersji 2304. W tej wersji nadal obsługiwane są wydruki w formie C#. To znaczy bez dokonania konwersji wzorców można z takich wydruków korzystać. Jednak od wersji 2304 po otwarciu wzorca do edycji, możliwy jest wyłącznie zapis do formy XML. Możliwe są także niezgodności w działaniu własnych lub zmienionych wydruków, które mogą wymagać ręcznej modyfikacji.

Wersja 2304 nie tylko udostępnia mechanizmy pod nowy format wzorców wydruków, ale także dostarcza wszystkie standardowe wydruki już w postaci skonwertowanej. Dzięki temu mogą stanowić wzorcowe przykłady do konwersji wydruków własnych w instalacjach enova365. Należy zwrócić uwagę, że konwersja formatu dotyczy nie tylko samych wzorców wydruków, ale także podraportów (nagłówków i stopek) oraz arkuszy styli.

Korzyści z konwersji

Konieczność zmiany sposobu przechowywania wzorców REPX wynikła z opisanej wyżej zmiany w środowisku .NET. Jednocześnie zmiana ta niesie ze sobą liczne korzyści wymienione poniżej, a opisane w szczegółach w kolejnych sekcjach dokumentacji.

Podział kompetencyjny : W nowej formie REPX wyraźniej oddzielona jest warstwa prezentacji wydruku (XML z kontrolkami graficznymi) od kodu logiki (osobny kod C#).

Selektywne kopiowanie : W przypadku potrzeby modyfikacji jedynie elementów graficznych wydruku, nie trzeba kopiować (a potem samodzielnie utrzymywać) kodu logiki, która zmienić się może w każdej kolejnej wersji oprogramowania.

Modularność : W nowej formie REPX fragmenty kodu logiki można współdzielić między wydrukami. Jeden wzorzec może wykorzystywać kilka różnych komponentów C# (tzw. *snippet-*

ów), z których każdy stanowi moduł ogólnego użytku w różnych wzorcach wydruków. *snippet* jest kodem logiki wydruku, a zarazem źródłem danych.

Brak skryptów : Pozbycie się skryptów uznanych za mechanizm przestarzały i posiadający luki bezpieczeństwa, przy jednoczesnym zachowaniu możliwości dynamicznej manipulacji wydrukiem za pomocą kodu C#.

Nowoczesna składnia : Do kompilacji skryptów w wewnętrznych mechanizmach DevExpress używany był wcześniej kompilator akceptujący C# w wersji 4.0, a zatem nie obsługujący nowoczesnych i powszechnych dzisiaj konstrukcji składniowych języka C# (operator strtzałki, interpolacja *string*-ów i wiele innych).

Kompatybilność

Ogólny mechanizm wykonywania wydruków z wzorców zapisanych w postaci C# został zachowany dla zgodności wstecz. Dotyczy to zarówno wersji enova365 opartej na .NET Framework, jak i wersji działającej .NET. W obu wersjach wzorce C# traktowane są jako przestarzałe, zalecamy ich konwersję do nowego formatu.

Podczas edycji wzorca w wersji Framework nadal możliwy jest także zapis do starej postaci kodu C#, obok zalecanej postaci XML. Natomiast w wersji .NET, z przyczyn opisanych powyżej, możliwy jest już tylko zapis wzorców do XML.

Mimo zachowania mechanizmu wykonywania wydruków w formie C#, w konsekwencji opisanych zmian w `repx` mogą pojawić się niezgodności, wymagające ingerencji wdrożeniowej.

Wycofanie `XtraReportSerialization` : Ponieważ wydruki przechowywane są teraz w postaci XML, przestały istnieć klasy reprezentujące wydruki, wcześniej przechowywane w przestrzeni nazw `XtraReportSerialization`. Odwołania to klas w tej przestrzeni wymagają zamiany na adekwatne klasy, reprezentujące logikę raportu. Powyższa zmiana dotyczy zarówno wzorców wydruków, jak i podraportów (nagłówków i stopek) i styli.

Brak typu reprezentującego wydruk : Ponieważ wzorce wydruków nie są już reprezentowane kodem C#, nie posiadają swojej klasy ani typu. Tym samym wszelkie konstrukcje umożliwiające wcześniej identyfikację wzorca przez typ klasy oznaczone zostają jako *obsolete*. Dotyczy to w szczególności mechanizmu `ReportResult` i własności `TemplateType`.

Kod logiki raportu

Poza warstwą wizualną zdefiniowaną w kontrolkach, wzorce wydruków posiadają często dodatkową logikę, opisaną w tzw. **skryptach**. Oto kilka typowych przykładów zastosowania skryptów:

- dynamiczna zmiana widzialności lub prezentacji elementów graficznych wydruku zależnie od podanych przez użytkownika parametrów wydruku lub wartości wyliczonych dynamicznie (np. zmiana koloru tekstu, ukrycie lub pokazanie kolumny w tabeli itp.)
- definicja klasy parametrów podawanych w oknie przy tworzeniu wydruku (nie jest to zalecany sposób przechowywania klasy parametrów, ale enova365 obsługuje go dla zachowania zgodności wstecz)
- definicje metod *event*-ów elementów wydruku - `OnBeforePrint`, `OnAfterPrint` i wiele innych

Więcej o zastosowaniu skryptów w wydrukach `XtraReports` można poczytać na stronach DevExpress poświęconych raportowaniu.

Czysto biznesowa logika powinna raczej znajdować się w modułach (w dodatkach, w workerach), a nie w raporcie. Jednak na poziomie wzorca wydruku potrzebujemy często

dodatkowego kodu, który powiąże logikę biznesową z warstwą wizualną wydruku, w celu uzyskania dynamicznie zmieniającego się wydruku zależnie od parametrów i innych wartości biznesowych. Bez możliwości tworzenia elementów dynamicznych każda wariacja wymagałaby klonowania całego wydruku z jedną drobną zmianą.

Logika zapisana w skryptach stanowi zatem integralną część wzorca wydruku. W wielu przypadkach wydruk nie może funkcjonować bez takiego dodatkowego kodu. Z drugiej jednak strony wyposażenie wydruku w praktycznie dowolny kod jest potencjalnie niebezpieczne. Stanowi podatność, która na audytach bezpieczeństwa systemów informatycznych określana jest mianem *formula injection vulnerability*. Z tego powodu DevExpress stopniowo wycofuje się ze skryptów, sugerując ich zamianę na wyrażenia. Skrypty, choć utrzymywane dla zgodności wstecz, traktowane są przez DevExpress raczej jako przeżytek technologiczny. Nie są rozwijane, do ich kompilacji stosowany jest przestarzały kompilator, który nie akceptuje nowszych konstrukcji składniowych języka C#. Można się spodziewać, że skrypty zostaną z czasem całkowicie wycofane. Jednocześnie wiemy, że nie każdą logikę można opisać w języku wyrażeń, a kod C# powiązany z wzorcem może być konieczny.

Począwszy od wersji 2304 wzorce wydruków w nowej formie nie posiadają skryptów w dotychczasowej postaci. Dzięki temu nie posiadają też wspomnianej podatności bezpieczeństwa i innych ograniczeń związanych z kodem zagnieżdżonym wewnątrz raportu. Wciąż jednak możliwe jest opisywanie logiki wydruku za pomocą kodu. Kod ten jednak jest przechowywany i utrzymywany inaczej, niż stosowane wcześniej skrypty. Szczegóły techniczne konstrukcji wzorców wydruków opisują kolejne sekcje.

Konstrukcja wzorców do wersji 2212

Format repx

Wzorce wydruków REPX zapisywane są do plików o rozszerzeniu `.repx`. To samo rozszerzenie używane jest do wzorców w formacie C# i XML. Tutaj omawiamy wzorzec `repx` w postaci kodu C#. Kod klasy wydruku ma następującą konstrukcję:

```
/// <XRTypeInfo>
/// (...komentarze zawierające XML z metadanymi)
/// </XRTypeInfo>
namespace XtraReportSerialization {

    public class MyReport : DevExpress.XtraReports.UI.XtraReport {
        /* deklaracje elementów */
        private DevExpress.XtraReports.UI.DetailBand band;
        private DevExpress.XtraReports.UI.XRLabel label;
        private DevExpress.XtraReports.UI.CalculatedField field;
        ...

        /* konstruktor */
        public MyReport() {
            this.InitializeComponent();
        }

        /* zasoby (w typ wszystkie skrypty) zapisane w postaci base64 */
        private System.Resources.ResourceManager resources {
            get {
```

```
                if (_resources == null) {
                    this._resources = new
DevExpress.XtraReports.Serialization.XRResourceManager("zsrvvgEAAACRAAAAbFN5c3Rl ... ");
            }

        /* inicjalizator wołany w konstruktorze */
        private void InitializeComponent() {
            this.band = new DevExpress.XtraReports.UI.DetailBand();
            this.label = new DevExpress.XtraReports.UI.XRLabel();
            this.field = new DevExpress.XtraReports.UI.CalculatedField();
            ...
            this.ScriptsSource = resources.GetString("$this.ScriptsSource");
            ...
        }
    }
}
```

Cały kod klasy wydruku generowany jest automatycznie za pomocą mechanizmu `CodeDOM`. Podobną postać otrzymamy otwierając wzorzec w edytorze i wybierając opcję zapisu **Zapisz do pliku repx**.

W standardowej formie wzorca `repx` kod skryptów zakodowany jest w postaci `base64` w zasobach wzorca (*property* `resources`). Podczas inicjalizacji obiektu wydruku kod skryptów jest podstawiany pod *property* `ScriptsSource`:

```
this.ScriptsSource = resources.GetString("$this.ScriptsSource");
```

Skrypty nie występują zatem jako jawny fragment kodu w klasie wydruku. Nawet jeśli kompletny kod klasy wydruku przechowywany byłby jako element projektu C# (np. w dodatku enova365), to kod logiki wydruku w tej standardowej formie nie byłby w ogóle widoczny. Oznacza to, że nie moglibyśmy wyszukać w projekcie symboli używanych w tym kodzie, VisualStudio lub inne IDE nie byłoby w stanie znaleźć referencji do logiki biznesowej użytej w logice wydruku itd.

Przyjrzyjmy się teraz mechanice działania skryptów. Kontrolki definiowane są jako prywatne pola w klasie wydruku - `band`, `label`, `field` i inne. Kod skryptów może manipulować kontrolkami, posługując się symbolami tych pól. Dla powyższych przykładów pól `label` i `field`, kod wyliczający dynamiczne wartości może wyglądać tak:

```
private void label_BeforePrint(object sender, System.Drawing.Printing.PrintEventArgs e) {
    ...
    label.Text = "dynamicznie wyliczana wartość etykiety";
}

private void field_GetValue(object sender, DevExpress.XtraReports.UI.GetValueEventArgs e) {
    ...
    e.Value = "wartość pola wyliczanego";
}
```

W formacie `repx` w klasie wydruku nie ma metod `label_BeforePrint()` i `field_GetValue()`. Metody te są ukryte w zakodowanych zasobach. W kodzie inicjalizującym komponenty znajduje się jedynie

przypisanie nazw metod zdefiniowanych w skryptach do *property* `Scripts`:

```
// this to inicjalizowana instancja wydruku
this.label = new DevExpress.XtraReports.UI.XRLabel();
this.field = new DevExpress.XtraReports.UI.CalculatedField();

// kontrolki wiązane są z nazwami metod ze skryptów
this.label.Scripts.OnBeforePrint = "label_BeforePrint";
this.field.Scripts.OnGetValue = "field_GetValue";
```

Widać więc, że metody *event*-ów ze skryptów kompilowane są niezależnie od klasy wydruku. Ich uruchomienie nie wykorzystuje *event*-ów, tylko przypisuje kontrolkom nazwy odpowiednich metod ze skryptu.

Instancja klasy `MyReport` tworzona jest tylko w wewnętrznych mechanizmach DevExpress, a użytkownik korzystający z wydruku w standardowy sposób i tak działa na instancji klasy bazowej `XtraReport`, a nie instancji swojej klasy `MyReport`. Powiązanie symboli wykorzystywanych w wydruku i w skryptach z elementami efemerycznego obiektu `MyReport` odbywa się przez refleksję. Obiekt raportu otrzymany po wczytaniu wzorca nie posiada żadnych pól zadeklarowanych w kodzie widocznych na powyższych przykładach (`band`, `label`, `field`). Nie dlatego, że są one prywatne, tylko dlatego, że te pola nie istnieją w binarnej strukturze obiektu.

Płynie stąd wniosek, że format `repx`, choć składniowo stanowi kompletną klasę C#, jest jedynie formą serializacji wzorca, a nie klasą obiektów, faktycznie wykorzystywaną do tworzenia wydruków. Inaczej mówiąc kod wzorca `repx` nie jest adekwatną reprezentacją obiektu, który otrzymamy po jego wczytaniu.

Wadą dynamicznej kompilacji skryptów w standardowych wzorcach `repx` jest to, że wykorzystywany przez DevExpress kompilator nie akceptuje wygodnych i powszechnie stosowanych dzisiaj konstrukcji składniowych, takich jak:

- operator `=>`
- operator `?`
- operator `??`
- formatowanie stringów za pomocą `$` i inne.

Przez to kod skryptów jest nie tylko niedostępny w klasie wydruku (dostępny tylko w edytorze), ale także niewygodny w tworzeniu i utrzymywaniu.

Format repx.cs

Od początku obsługi wydruków REPX enova365 posiada wbudowany mechanizm, który dynamicznie konwertuje standardowe wzorce wydruków do formatu `repx.cs`, likwidując pewne niedogodności standardu `repx`. Zapis `repx.cs` można otrzymać otwierając wzorzec w edytorze i wybierając opcję zapisu **Zapisz jako kod repx.cs**. Ten format to także kod klasy C# dziedziczącej z `DevExpress.XtraReports.UI.XtraReport`, podobnie jest w formacie `repx`. Jednak `repx.cs` tym się różni od standardowego zapisu `repx`, że kod logiki wydruku (skrypty) dostępny jest w czytelnej, odkodowanej formie, w ramach klasy wydruku. Kod `repx.cs` posiada sekcje wydzielone za pomocą charakterystycznych znaczników:

- `// Report Script Start` - początek kodu logiki (skryptów)
- `// Report Script End` - koniec kodu logiki
- `#region Report definition` - początek definicji wydruku
- `#endregion` - koniec definicji wydruku

Wzorzec wydruku w formie `repx.cs` ma następującą konstrukcję:

```
namespace XtraReportSerialization {
    /* wygenerowana przez konwerter lista using-ów */
    using System;
    using Soneta.Business;
    using Soneta.Kadry;
    ...

    public class MyReport : DevExpress.XtraReports.UI.XtraReport {

        // Report Script Start
        // ... czytelny kod logiki wydruku wyciągnięty ze skryptów
        // Report Script End

        /* wygenerowana przez konwerter metoda inicjalizacji event-ów */
        private void InitializeEvents() {
            BeforePrint += MyReport_BeforePrint;
            label.BeforePrint += label_BeforePrint;
            field.GetValue += field_GetValue;
            ...
        }

        # region Report definition

        /* deklaracje elementów raportu, jak w standardowym repx */
        private DevExpress.XtraReports.UI.DetailBand band;
        private DevExpress.XtraReports.UI.XRLabel label;
        private DevExpress.XtraReports.UI.CalculatedField field;
        ...

        /* zmodyfikowany konstruktor z dodanym wywołaniem InitializeEvents() */
        public MyReport() {
            this.InitializeComponent();
            this.InitializeEvents();
        }

        /* ... zasoby i definicja InitializeComponent(), podobnie do .repx */

        # endregion
    }
}
```

Kod logiki wydruku umieszczony między komentarzami `// Report Script Start` oraz `// Report Script End` jest dostępny w klasie wydruku, a metody *event*-ów łączone są z kontrolkami w wygenerowanej automatycznie metodzie `InitializeEvents()`. Oznacza to, że skrypty w rozumieniu specjalnego pola `ScriptsSource` z kodem w tej formie nie istnieją i nie są potrzebne. Instancja wydruku utworzona z takiej klasy z punktu widzenia mechanizmów DevExpress w ogóle nie posiada skryptów:

```
var report = new MyReport();
Debug.Assert(report.ScriptsSource == null);
```

Metody `label_BeforePrint()` oraz `field_GetValue()` są zwykłymi metodami w klasie wydruku, mogą zatem odwoływać się do dowolnych pól instancji (do powiązania skryptu z wydrukiem nie jest potrzebna refleksja). Natomiast powiązanie metod z *event*-ami, w przeciwieństwie do `repx`, odbywa się wprost, w wygenerowanej przez konwerter metodzie `InitializeEvents()`. Pod tym kątem zapis `repx.cs` jest bardziej przejrzysty. Pozwala na utrzymywanie kodu logiki wydruku w czytelnej postaci, w systemie kontroli wersji, jak każdy inny kod logiki biznesowej.

Jeśli mamy dostęp do narzędzi pozwalających na kompilację wydruków (czyli odpowiednią licencję DevExpress), wtedy wzorzec w postaci `repx.cs` możemy włączyć w projekt modułu enova365. Tak osadzone są wszystkie standardowe wydruki dostarczane z enova365. Klasa wydruku `MyReport` zapisana w `repx.cs` jest wtedy kompilowana do *assembly* i funkcjonuje jak każdy inny typ w tym *assembly*. Instancjonowanie wydruku w enova365 odbywa się przez wywołanie konstruktora klasy wydruku. Ponieważ wzorzec wydruku jest jednoznacznie identyfikowany przez typ `MyReport`, w większości przypadków instancja wydruku tworzona jest przez wywołanie `System.Activator.CreateInstance(<typ wydruku>)`.

Aby wzorzec wydruku osadzony w module enova365 był widziany w menu wydruków, powinien zostać zarejestrowany na poziomie *assembly*. Do rejestracji służy atrybut `DxReport`, zwyczajowo umieszczany w pliku `AssemblyInfo.RepxReport.cs`.

```
[assembly: DxReport(
    typeof(MyRow),              // typ obiektów dla których raport ma być dostępny w menu
    "Moje wydruki/Mój wydruk", // oczekiwana ścieżka w menu wydruków
    typeof(MyReport)            // typ klasy raportu
]
```

To wystarczy, by moduł enova365 dostarczał wydruk dostępny automatycznie w menu wydruków dla danego rodzaju obiektów biznesowych. Natomiast kompletna klasa wydruku (wraz z logiką raportu), tak jak klasy biznesowe, kompilowana jest wraz z projektem modułu enova365.

Uwaga!

Ten rodzaj rejestracji wydruku REPX we własnym rozwiązaniu wymaga referencji do bibliotek DevExpress zawierających implementację klas związanych z wydrukami. Są to biblioteki licencjonowane. Nie można w tym celu skorzystać z bibliotek DevExpress dostarczanych z enova.

Raporty w bazie

Kiedy standardowy raport `repx.cs` chcemy zmodyfikować w programie, musi nastąpić skopiowanie raportu do bazy danych. Nie dałoby się inaczej zmodyfikować kodu skompilowanego do *assembly* dostarczanego z programem. Raport skopiowany do bazy danych na potrzeby zmian przykrywa raport standardowy o tej samej nazwie, zarejestrowany w *assembly*.

Aby umożliwić modyfikację zarówno warstwy wizualnej raportu jak i jego logiki, do kopii raportu przenoszony jest cały kod logiki w postaci skryptu. Do bazy danych trafia wzorzec zapisany w standardowej formie `repx`, którą edytor wydruków potrafi otworzyć i udostępnić do edycji. Raporty skopiowane do bazy lub utworzone w bazie (bez rejestracji w *assembly*) posiadają wszystkie cechy własności opisane wyżej dla formatu `repx`.

Konstrukcja raportów od wersji 2304

Wymuszona przez ewolucję .NET zmiana w obsłudze wzorców REPX, daje szansę na pozbycie się ograniczeń, jakie posiada format `repx` w postaci C#. Standardowe raporty dostarczane przez enova365 od wersji 2304 przechowywane są w postaci XML. Jest to zgodne z zaleceniami DevExpress i daje możliwość utrzymywania wspólnego formatu wzorców wydruków dla wersji .NET Framework i Core.

Format .repx

W plikach z rozszerzeniem `.repx` przechowywany jest tutaj XML. Jest to postać zapisu wzorca funkcjonalnie równoważna z wcześniejszą formą C#, ale nie wymaga wycofanej z .NET technologii CodeDOM.

Wzorzec w postaci XML staje się domyślną formą zapisu, zastępującą wcześniejszą formę C#, przy czym przy zapisie do pliku wzorzec posiada to samo rozszerzenie `.repx`.

Wzorce w postaci XML definiują elementy wizualne raportu, analogicznie jak deklaracje prywatnych pól w klasie raportu w starszym zapisie `repx` i `repx.cs`.

```
<?xml version="1.0" encoding="utf-8"?>
<XtraReportsLayoutSerializer SerializerVersion="20.2.7.0" Ref="1"
ControlType="DevExpress.XtraReports.UI.XtraReport, ..." Name="Cennik" ...>
    <Bands>
        <Item1 Ref="2" ControlType="TopMarginBand" Name="TopMarginBand" HeightF="200" Dpi="254"
/>
        <Item2 Ref="3" ControlType="DetailBand" Name="TreśćRaportu" HeightF="0" Dpi="254" />
        <Item3 Ref="4" ControlType="BottomMarginBand" Name="BottomMarginBand" HeightF="200"
Dpi="254" />
...
      <Controls>
        <Item1 Ref="13" ControlType="XRTable" Name="table1" AnchorHorizontal="Both"
TextAlignment="MiddleCenter" SizeF="1701.8,38.1" LocationFloat="0,0" Dpi="254">
          <Rows>
            <Item1 Ref="14" ControlType="XRTableRow" Name="tableRow1" Weight="11.5" Dpi="254">
...
<ComponentStorage>
    <Item1 Ref="42" Name="BusinessContext" ... />
    <Item2 Ref="9" Name="BusinessSource" ... />
    <Item3 Ref="0" Name="BusinessSourceContext ... />
    <Item4 Ref="43" Name="Snippet"
SnippetTypeName="Soneta.Handel.Reports.CennikSnippet,Soneta.Handel.Reports" />
</ComponentStorage>
```

Cała konstrukcja wzorca to drzewo kontrolek budowane dynamicznie podczas tworzenia instancji wydruku. Sam obiekt `XtraReport` również jest kontrolką, to znaczy na pewnym poziomie w hierarchii dziedziczenia obiekt raportu posiada te same klasy bazowe, co etykiety, bandy, tabelki i inne kontrolki. Dzięki temu wszystkie obiekty raportu mają wspólne zachowanie dotyczące *event*-ów.

Do wzorca podpięte są źródła danych specyficzne dla Soneta (element `ComponentStorage`). Jednym z komponentów raportu jest `Snippet`, czyli klasa opisująca logikę raportu. Klasa ta dla standardowych wydruków przechowywana jest w kodzie enova365, a dla wydruków w rozwiązaniach partnerskich - w dodatkach. W ten sposób wzorzec XML zapisany jako `repx` powiązany jest z kodem logiki wydruku (*snippet*-em), zapisanym jako `repx.cs`.

Poniżej przykład powiązania raportu XML z kodem dwóch *snippet*-ów. `Snippet1` to standardowy *snippet* pochodzący z modułu handel. Atrybut `SnippetTypeName` wskazuje na jego typ, czyli `Soneta.Handel.Reports.SprzedazSnippet`. `Snippet2` to dodatkowy kod dodany użytkownika, tj. przechowywany w bazie. Atrybut `SnippetFile` wskazuje na nazwę pliku z kodem do dynamicznej kompilacji.

```
<ComponentStorage>
    ...
    <Item5 Ref="572"

ObjectType="Soneta.Business.UI.DxReports.ReportSnippetComponent,Soneta.Business.UI.DxReports"
        Name="Snippet1"
        SnippetTypeName="Soneta.Handel.Reports.SprzedazSnippet,Soneta.Handel.Reports" />
    <Item6 Ref="573"

ObjectType="Soneta.Business.UI.DxReports.ReportSnippetComponent,Soneta.Business.UI.DxReports"
        Name="Snippet2"
        SnippetFile="ExtraSnippet" />
</ComponentStorage>
```

Format .repx.cs

W plikach z rozszerzeniem `.repx.cs` przechowywany jest kod C# zarządzający wydrukiem. W odróżnieniu od poprzedniego rozwiązania, plik `.repx.cs` nie jest samodzielnym wzorcem wydruku. Jest kodem wydzielonym z wzorca, pełniącym tę funkcję, którą dawniej pełniły skrypty. Sam wzorzec wydruku przechowywany jest w postaci XML.

Dawniej klasa zapisywana w postaci `repx.cs` dziedziczyła ze standardowej reprezentacji wydruku, czyli z `DevExpress.XtraReports.UI.XtraReport`. W nowym rozwiązaniu klasa ta dziedziczy z `Soneta.Business.UI.DxReports.ReportSnippet`.

```
namespace MyReports
{
    public class MyReportSnippet : ReportSnippet
    {
        /* ew. klasa parametrów (bez zmian) */
        public class MyReportParams : ContextBase
        {
            public MyReportParams(Context cx) : base(cx) { ... }
        }
```

```
        /* deklaracje elementów do których odwołuje się kod event-ów */
        [DxBind] private DevExpress.XtraReports.UI.DetailBand band;
        [DxBind] private DevExpress.XtraReports.UI.XRLabel label;
        [DxBind] private DevExpress.XtraReports.UI.CalculatedField field;
        ...

        /* definicje event-ów */
        [DxBind]
        private void label_BeforePrint(object sender, CancelEventArgs e) {
            ...
            label.Text = "dynamicznie wyliczana wartość etykiety";
        }

        [DxBind]
        private void field_GetValue(object sender, DevExpress.XtraReports.UI.GetValueEventArgs e)
{
            ...
            e.Value = "wartość pola wyliczanego";
        }
    }
}
```

Za powiązanie pól klasy *snippet* (tutaj `bind`, `event`, `field`) odpowiada atrybut `[DxBind]`. To rozwiązanie pozwala na zachowanie kluczowej funkcjonalności, jaką jest oprogramowanie wzorców wydruków, jednocześnie umożliwia wycofanie się ze skryptów w dotychczasowej postaci ze względów opisanych w poprzednich sekcjach.

W przykładzie widać, że atrybut `[DxBind]` stosujemy w dwóch przypadkach:

- przy deklaracji pól pochodzących z raportu
- przy sygnaturze metod *event*-ów

Atrybut przy deklaracji pola gwarantuje, że enova365 danemu polu przypisze dynamicznie odpowiednią wartość pochodzącą z wzorca raportu. Dzieje się to przy inicjalizacji wydruku, tuż przed jego wykonaniem. Atrybut przy metodzie powoduje, że metoda jest przypisywana do odpowiedniego *event*-u odpowiedniej kontrolki w raportcie.

Deklaracje symboli pochodzących z raportu (w przykładzie `band`, `label`, `field`) są zadeklarowane w klasie `MyReportSnippet`. Dzięki temu w tej klasie można zdefiniować metody *event*-ów (`BeforePrint`, `GetValue`), które w trakcie wykonania wydruku będą te elementy dynamicznie przekształcać.

Należy zwrócić uwagę na to, by elementy te posiadały odpowiedniki we wzorcu XML, który kod zasila. Elementy `band`, `label`, `field` nie są inicjalizowane w tej klasie, ich wartości pochodzą z obiektu wydruku. Zanim uruchomione zostaną metody *event*-ów, enova365 automatycznie przypisze do tych pól odpowiednie wartości z instancji wydruku.

Klasa `DxBindAttribute` ma następującą postać:

```
namespace Soneta.Business.UI.DxReports {
    [AttributeUsage(AttributeTargets.Field | AttributeTargets.Method, AllowMultiple = true)]
    public class DxBindAttribute : Attribute {
        public string Name { get; set; }
```

```
        public string EventName { get; set; }
        public bool Required { get; set; } = true;
    }
}
```

Domyślnie w obiekcie wydruku wyszukiwany jest element o tej samej nazwie, co element w klasie logiki wydruku. Nazwy metod *event*-ów traktowane są jako złączenie nazwy pola z nazwą zdarzenia. Na przykład, jeśli użyjemy

```
[DxBind]
private void MyControl_BeforePrint() { ... }
```

, za nazwę kontrolki zostanie przyjęte `MyControl` a za nazwę *event*-u `BeforePrint`. Można to kontrolować za pomocą własności `Name` oraz `EventName` w atrybucie `[DxBind]`. Dzięki temu z jedną kontrolką można powiązać więcej niż jeden *event*.

```
[DxBind(Name="MyControl1", EventName="BeforePrint")]
[DxBind(Name="MyControl2", EventName="BeforePrint")]
[DxBind(Name="MyControl3", EventName="BeforePrint")]
private void DoBeforePrint(object sender, CancelEventArgs e) { ... }
```

Pewna oboczność dotyczy *event*-ów głównego obiektu raportu. Jeśli dany event dotyczy raportu, należy wskazać nazwę `"Report"`.

```
    [DxBind(Name="Report", EventName="DataSourceRowChanged")]
    private void ReportName_DataSourceRowChanged(object sender,
DevExpress.XtraReports.UI.DataSourceRowEventArgs e) {...}

    [DxBind(Name="Report", EventName="BeforePrint")]
    private void ReportName_BeforePrint(object sender, CancelEventArgs e) {...}
```

Należy w związku z tym stosowania nazwy `"Report"` dla innych kontrolek raportu.

Należy zauważyć, że w klasie *snippet*-a zadeklarowane pola nie są inicjowane wartością (nie ma przypisań do tych pól). Brak inicjalizacji pól może powodować ostrzeżenie kompilatora, że pole posiada zawsze początkową wartość `null`. Ostrzeżenie można zignorować (np. wyciszyć za pomocą `#pragma warning disable`), bo pole zostanie poprawnie zainicjalizowane dzięki obecności atrybutu `[DxBind]`.

Istotną konsekwencją opisanych zmian jest to, że wzorzec nie ma już reprezentacji w postaci klasy C#. Tym samym nie można go zidentyfikować za pomocą typu (`System.Type`), ani utworzyć w kodzie instancji tego typu wywołując konsturktor klasy raportu lub używając `System.Activator.CreateInstance()`, jak to miało miejsce w starszym rozwiązaniu. Obiekt wydruku pozostaje instancją klasy `DevExpress.XtraReports.UI.XtraReport`, ale jest wczytywany z wzorca `repx` w postaci XML, a zasilany kodem z `repx.cs`.

W konsekwencji zmienia się sposób rejestracji wydruków wbudowanych w moduły (dodatki). Trzecim parametrem przekazywanym wzorzec XML:

```
[assembly: DxReport(
    typeof(MyRow),             // typ obiektów dla których wydruk ma być dostępny w menu (bez
zmian)
    "Moje wydruki/Mój wydruk", // oczekiwana ścieżka w menu wydruków (bez zmian)
    "MyReport.repx",           // nazwa wzorca zamiast typu klasy, zamist typeof(MyReport)
]
```

Wycofanie własnych klas (typów) reprezentujących wydruki powoduje, że używana wcześniej przestrzeń nazw `XtraReportSerialization` przestaje istnieć, podobnie jak wszystkie symbole w tej przestrzeni:

- `XtraReportSerialization.Sprzedaz`
- `XtraReportSerialization.Pit1129`
- `XtraReportSerialization.UmowaZlecenie`
- itd.

Wszystkie te klasy posiadają odpowiedniki w nowym rozwiązaniu, przy czym odpowiedniki mają inny typ i inną nazwę klasy, np.:

- `XtraReportSerialization.Sprzedaz` -> `Soneta.Handel.Reports.SprzedazSnippet`
- `XtraReportSerialization.Pit1129` -> `Soneta.Deklaracje.Reports.Pit1129Snippet`
- `XtraReportSerialization.UmowaZlecenie` -> `Soneta.KadryPlace.Reports.UmowaZlecenieSnippet`
- itd.

Jeśli gdzieś w istniejących wydrukach lub podraportach (nagłówkach czy stopkach) istnieją odwołania do standardowych symboli mieszkających dawniej w przestrzeni nazw `XtraReportSerialization`, to wydruki takie wymagają zmiany. W większości przypadków jest to zmiana nazwy typu. Na przykład, jeśli korzystamy z własnego wydruku stworzonego dawniej na podstawie standardowego wzorca `Dokument/Dokument sprzedaży` (faktura VAT), to we wzorcu znajduje się prawdopodobnie taki kod:

```
public XtraReportSerialization.Sprzedaz.MyParametryWydruku Parametry { get; set; }
```

Będzie powodował błąd kompilacji, bo nie istnieje już typ `XtraReportSerialization.Sprzedaz`. Kod ten powinien zostać zmieniony na

```
public Soneta.Handel.Reports.SprzedazSnippet.MyParametryWydruku Parametry { get; set; }
```

Podobna sytuacja może się zdarzyć w wydrukach korzystających z podraportów (nagłówków, stopek i innych), które zostały dawniej skopiowane ze standardowych i zmodyfikowane, a przez to nie mogą być automatycznie aktualizowane z kolejnymi wersjami enova365. Przykłady starszego kodu w subraportach, który wymaga aktualizacji.

```
// nagłówek dokumentu handlowego
resourceManager = new ResourceManager("Soneta.Repx.Sprzedaz",
typeof(XtraReportSerialization.Sprzedaz).Assembly);

// podpisy dokumentu handlowego
var sub = MasterReport as XtraReportSerialization.Sprzedaz;
```

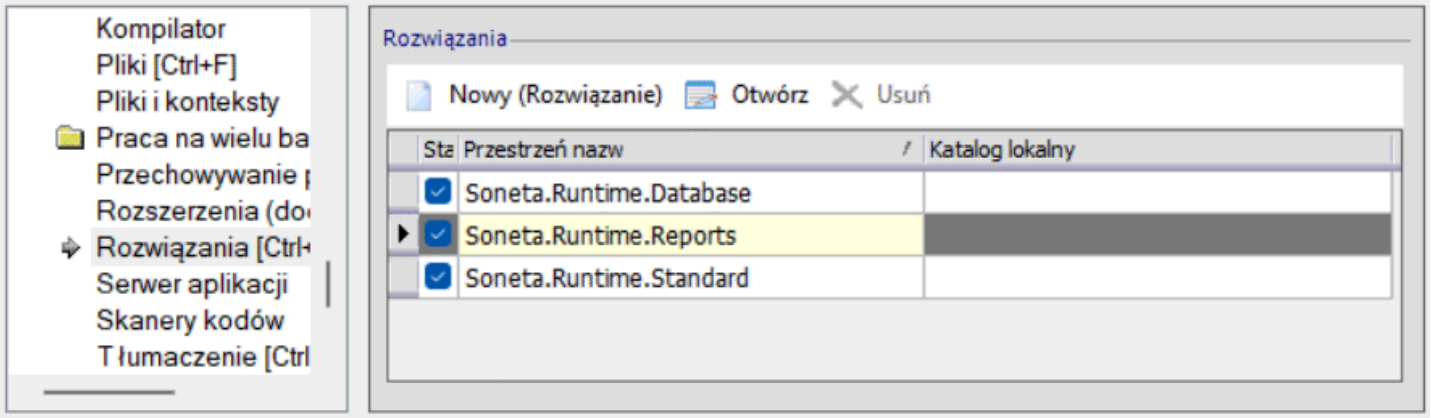
W przypadku podraportów zalecamy przywrócenie standardowych, ew. naniesienie odpowiednich zmian na nową postać standardowych nagłówków.

Raporty w bazie

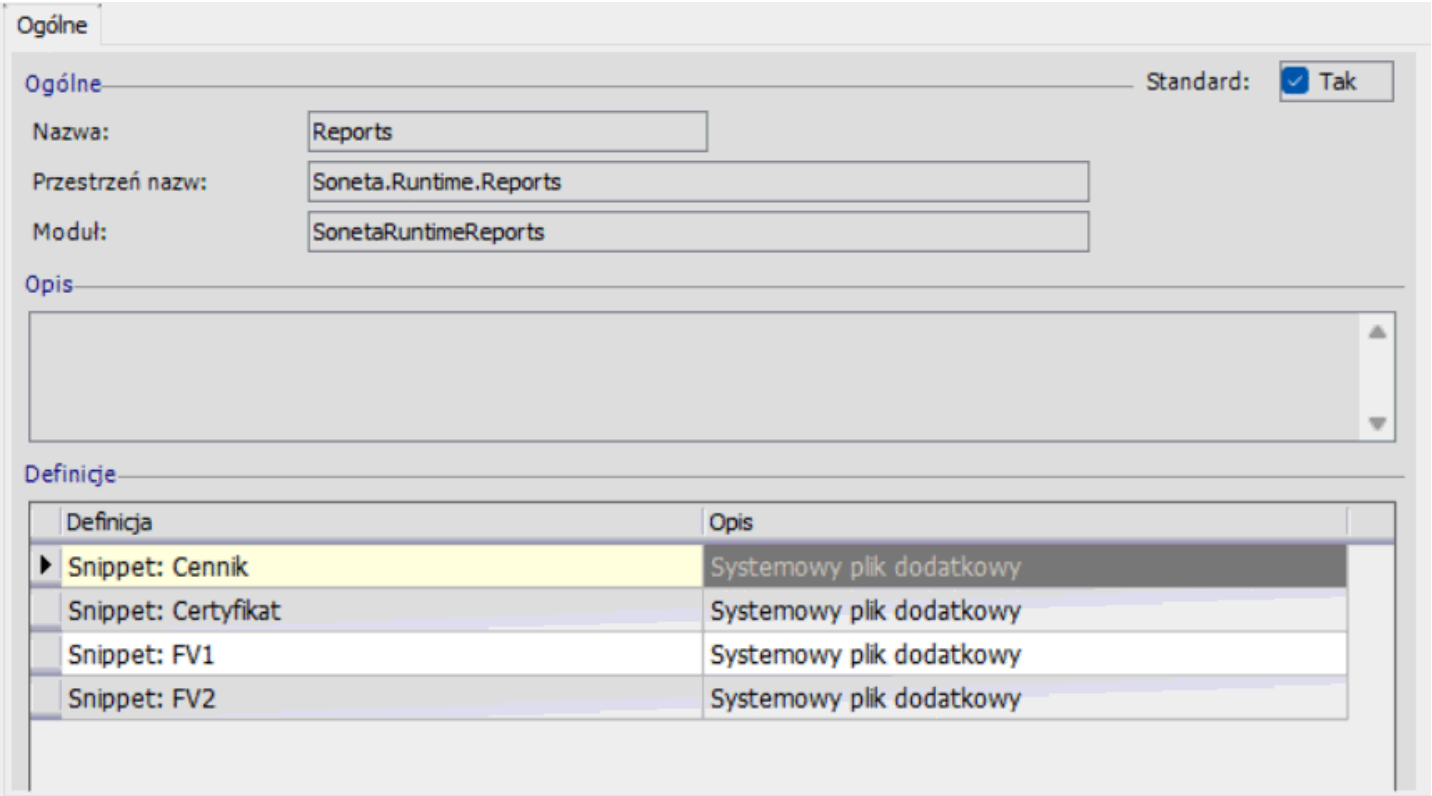
Własne wzorce wydruków dodawane ręcznie do list i formularzy, a także modyfikowane wzorce standardowe, kopiowane są do bazy danych. Podobnie jak w poprzednim rozwiązaniu, wzorzec umieszczony w bazie przechowywany jest w formie pliku `.repx`, przy czym od wersji 2304 jest to plik XML, a nie format C#. Natomiast kod logiki wydruku traktowany jest tutaj inaczej niż we wcześniejszym rozwiązaniu:

- albo pozostaje bez zmian - modyfikujemy wtedy tylko warstwę wizualną raportu, a kod zasilający pozostaje standardowy
- albo zostaje skopiowany do konfiguracji systemu i traktowany jako algorytm kompilowany dynamicznie, tak jak cechy, taski i inne algorytmy biznesowe

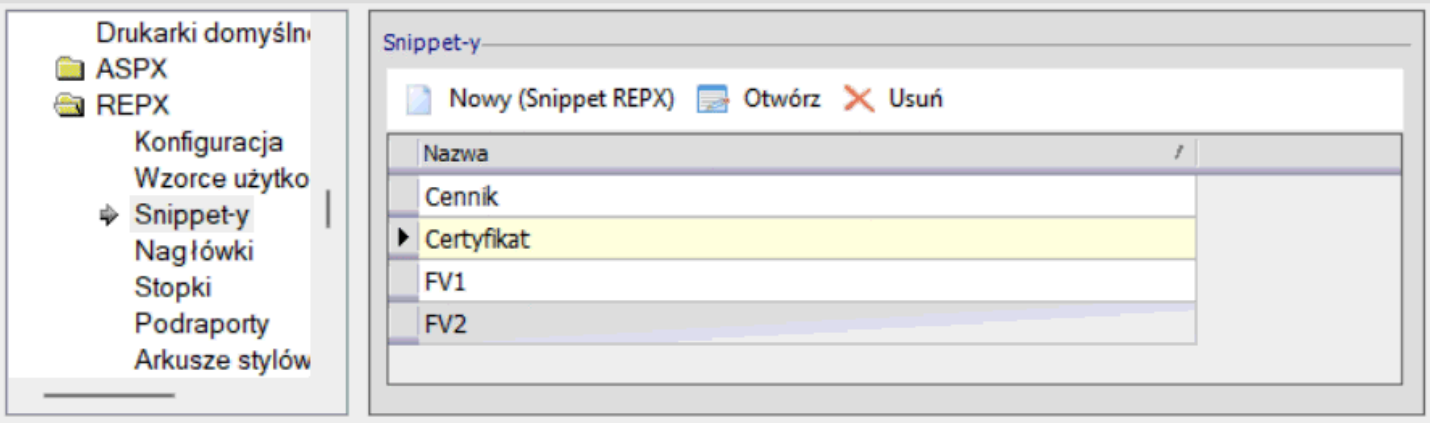
W poprzednim rozwiązaniu przy otwarciu wzorca do edycji kod logiki był zawsze w całości kopiowany do wzorca w postaci skryptu, przez co nie mógł być aktualizowany wraz z aktualizacjami enova365. Prowadziło to też do zwielokrotnienia podobnego kodu, bo na potrzeby każdej modyfikacji dowolnego wydruku kod standardowy w całości przenoszony był do wzorca. Przez to w wydrukach w instalacjach produkcyjnych osadzony jest kod, który jest wyłącznie pod opieką partnera, poza mechanizmami aktualizacji w ramach konwersji. Od wersji 2304 nie musimy kopiować kodu powiązanego z wydrukiem. Jeśli tego potrzebujemy, kod ten zostaje przeniesiony do algorytmu kompilowanego dynamicznie, w ramach projektu `Soneta.Runtime.Reports` (*Ustawienia* -> *Systemowe* -> *Rozwiązania* -> *Projekty*). Rozwiązanie (solucja):



W obrębie rozwiązania projekt o tej samej nazwie `Soneta.Runtime.Reports`:



Kod algorytmów związanych z wydrukami dostępny jest też w dedykowanej zakładce *Ustawienia -> Raportowanie -> REPX -> Snippety*.



Jeśli skopiowanie i modyfikacja kodu logiki wydruku jest konieczna, zalecamy dziedziczenie i rozszerzenie klasy reprezentującej *snippet* zamiast jej podmiany własną klasą. Kolejne wersje enova365 nie mogą zaktualizować zmodyfikowanego kodu wydruku przechowywanego jako algorytm zmodyfikowany wdrożeniowo, jednak modyfikacje funkcjonalne wykonane przez dziedziczenie łatwiej utrzymywać.

Korzystanie ze snippet-ów

W kodzie *snippet*-a można zdefiniować dowolną logikę, która potrzebna jest do oprogramowania wydruku. Może być to zarówno kod zarządzający konkretnymi kontrolkami raportu, jak i ogólniejszy kod wyliczający dane do prezentacji, powiązany z logiką biznesową, np. klasa parametrów, źródło danych i in.

W enova365 w wersji 2304 zachowany został dotychczasowy schemat, w którym jednemu wzorcowi wydruku odpowiada jeden *snippet*, np. wydruk `Sprzedaz` powiązany jest z `SprzedazSnippet`, wydruk `UmowaZlecenie` powiązany jest z `UmowaZlecenieSnippet` itd. Z czasem te powiązania mogą się zmieniać, bo nowy mechanizm nie zakłada tylko takiego powiązania. Dawniej skrypt należał do raportu, był z nim sztywno powiązany. Teraz *snippet* jest odrębnym komponentem. Jeden

raport może posiadać powiązania z kilkoma *snippet*-ami. Jeden *snippet* może zasilać kilka podobnych raportów.

Dziedziczenie

Standardowe *snippet*-y dostarczane z enova365 od wersji 2304 dziedziczą bezpośrednio z klasy bazowej `Soneta.Business.UI.DxReports.ReportSnippet`

```
/* klasa bazowa dla wszystkich snibppet-ów */
namespace Soneta.Business.UI.DxReports {
    public class ReportSnippet : IContextable, ISessionable {
        ...
    }
}

/* klasa kodu wydruku handlowego */
namespace Soneta.Handel.Reports {
    public class SprzedazSnippet : ReportSnippet {
        ...
    }
}
```

W rozwiązaniach wdrożeniowych można rozszerzyć hierarchię dziedziczenia, na przykład:

```
public class CustomReportSnippet : ReportSnippet {
    /* klasa ogólna, wspólna dla grupy wydruków */
}

public class LeafSnippet1 : CustomReportSnippet {
    /* klasa snippet-a powiązana z wydrukiem */
}

public class LeafSnippet2 : CustomReportSnippet {
    /* klasa snippet-a powiązana z wydrukiem */
}
```

Nie należy natomiast tworzyć własnych klas dziedziczących ze standardowych *snippet*-ów.

```
public class MySprzedazSnippet : SprzedazSnippet {
    /* to nie jest poprawne */
}
```

Klasy standardowych *snippet*-ów nie są *sealed*, więc taki kod się przekompiluje poprawnie. Jednak powiązanie `MySprzedazSnippet` z wydrukiem spowoduje najprawdopodobniej jego uszkodzenie, bo nie zostaną poprawnie powiązane metody *event*-ów opisane w `SprzedazSnippet` atrybutem `DxBind`. Co więcej, standardowe klasy *snippet*-ów działają na polach prywatnych, nie wystawiają metod wirtualnych do *override*, więc dziedziczenie po nich nie ma sensu. Możemy natomiast rozszerzyć funkcjonalność raportu dodając własną klasę, w swojej hierarchii dziedziczenia, niezależną od klasy *snippet*-a standardowego.

Parametry

Jeśli raport potrzebuje parametrów, najłatwiej zdefiniować klasę parametrów w kodzie wydruku, np.:

```
public class CustomReportSnippet : ReportSnippet {
    ...
        public class CustomParams : ContextBase {
                public CustomParams(Context context) : base(context) {}

        [Caption("Dodatkowy parametr")]
                public string CustomProperty { get; set; }
        }
}
```

Jest to o tyle dobre rozwiązanie, że cały kod związany z wydrukiem, również klasę parametrów, możemy utrzymywać w jednym miejscu. Zdefiniowanie klasy parametrów w kodzie wydruku posiada jednak pewną wadę. Kiedy chcemy uruchomić wydruk z workera i utworzyć klasę parametrów automatycznie, dostęp do typu klasy parametrów jest utrudniony. Nie wynika to z samej konstrukcji *snippet*-ów ale z tego, że z kodu kompiloweanego w ramach dodatku (poza enova) musimy się dostać do typu kompilowanego dynamicznie i przechowywanego w bazie danych.

Dawniej, kiedy klasa parametrów osadzona była w skrypcie, dostanie się do niej z workera nie było możliwe. Obecnie jest możliwe przy użyciu standardowych konstrukcji framework-a enova, ale wymaga użycia refleksji. Poniżej przykład odwołania do typu `CustomParams` zdefiniowanego w klasie `CustomReportSnippet`, który przechowywany jest w bazie danych.

```
[Action("Wydruk", Mode = ActionMode.SingleSession, Target = ActionTarget.Menu |
ActionTarget.Toolbar)]
public void Action(Context context)
{
    /* pobieramy z bazy obiekt przechowujący kod snippet-a */
    var sysFile = context.Session.GetBusiness()
        .SystemFiles.ByName["CustomReportSnippet"];

    /* pobieramy z sesji potrzebny typ, w tym miejscu może się uruchomić dynamiczna kompilacja */
    paramsType = context.Session.AssemblyCache.GetType(sysFile,
"Geeks.CustomReportSnippet+CustomParams");

    /* mając typ, tworzymy instancję i przekazujemy do kontekstu */
    context[paramsType] = Activator.CreateInstance(paramsType, context);
    var result = new ReportResult {
        Context = context,
        Format = ReportResultFormat.PDF,
        TemplateFileName = "Sprzedaz.repx",
        DataType = typeof(DokumentHandlowy)
    };
```

```
    ...
}
```

Tabela `SystemFiles` jest wykorzystywana do przechowywania kodu *snippet*-ów. Natomiast konstrukcja `Session.AssemblyCache.GetType` dotyczy dowolnego typu kompilowanego dynamicznie, nie tylko kodu wydruków.

Klasy związane ze snippet-ami

- `ReportSnippet` - klasa bazowa dla algorytmów zasilających wydruki REPX. Klasy dziedziczące z niej pełnią tę funkcję, którą dawniej miały osadzone w raportach skrypty.
- `ReportSnippetComponent` - wiąże obiekt raportu (`XtraReport`) ze *snippet*-em. Raport zawiera kolekcję komponentów, komponent daje dostęp do *snippet*-a. Kod wiązany jest z komponentem na dwa sposoby:
    - albo poprzez nazwę typu z którego powstanie instancja snippet-a, np. `Soneta.Handel.Reports.CennikSnippet` (w XML atrybut `SnippetTypeName`)
    - albo poprzez dowiązanie do obiektu zawierającego kod do dynamicznej kompilacji (w XML atrybut `SnippetFile`)
- `DxReportFile` obiekt biznesowy posiadający reprezentację w bazie danych w postaci tabeli `SystemFiles`, przechowujący kod wydruków do dynamicznej kompilacji. Wpis w tabeli `SystemFiles` dodawany jest wtedy, gdy kod zasilający wydruk jest kopiowany ze swej oryginalnej postaci do bazy danych na potrzeby edycji. Do tabeli `SystemFiles` można też dodawać własne *snippet*-y tworząc nowy wpis i wypełniając go kodem (*Ustawienia -> Raportowanie -> Snippet-y*).

Konwersja manualna

Tutorial z konwersją przykładowego raportu znajduje się tutaj W ogólności konwersja wydruków z postaci C# do XML jest zawsze podobna i składa się z kilku kroków:

- podmiana formatu C# na XML w pliku `.repx`
- utworzenie poprawnej klasy *snippet*-a w miejsce dotychczasowego skryptu raportu
- powiązanie raportu ze *snippet*-em
- rejestracja raportu w dodatku lub osadzenie w bazie

Zamiana C# na XML : Otwieramy wydruk do edycji w designerze (w wersji 2304 lub wyżej) i zapisujemy plik `.repx` (xml). Jeśli raport nie zawiera skryptów, to jest to praktycznie koniec konwersji. Sposób postępowania ze skryptem opisany poniżej. Należy obejrzeć wynikowy XML i popatrzyć, czy nie zostały w nim artefakty serializacji, np. odwołania do lokalnych ścieżek na dysku.

Przeniesienie skryptu : Jeśli zapiszemy plik .repx nie usuwając skryptu z raportu to ten skrypt zostanie umieszczony w wynikowym pliku w atrybucie `ScriptSource`, w głównym elemencie XML-a. Tak osadzony skrypt nadal będzie działał, przykrywając ew. implementacje metod dostarczone w *snippet*. Skrypt osadzony w XML-u nie jest to dobrym rozwiązaniem, z przyczyn szeroko opisanych tutaj. Dlatego skrypt należy wyciąć z raportu przed zapisaniem pliku `.repx`. Kod skryptu zapisujemy jako `.repx.cs`.

Utworzenie klasy snippet : Kod skryptu wyciągnięty z raportu nie jest samodzielną klasą, jest zbiorem metod *event*-ów działających na kontrolkach raportu. Należy go zmodyfikować, aby powstała klasa *snippet*-a. W tym celu w kodzie `.repx.cs` owijamy kod skryptów przestrzenią nazw oraz klasą:

```
namespace Soneta.MyModule.Reports {
    public class MyReportSnippet : ReportSnippet {
        /* tutaj kod skryptu wyciętego z raportu */
    }
}
```

Ważne jest tutaj zachowanie przyjętych konwencji:

- klasa musi posiadać suffix `Snippet`
- klasa musi dziedziczyć z `ReportSnippet`
- plik z klasą musi się nazywać `MyReport.repx.cs`, ew. `MyReportSnippet.repx.cs`

Powiązanie z elementami raportu : Do każdej metody *event*-ów (np. `BeforePrint`) dodajemy atrybut `DxBind`:

```
[DxBind]
MyObject_BeforePrint() { /* tu kod bez zmian */ }
```

Wg konwencji, metody *event*-ów są dwuczłonowe. Pierwszy człon to nazwa obiektu, drugi to nazwa *event*-u. Jeśli z nazwy metody nie wynika jakiego obiektu i jakiego *event*-u dotyczy, to trzeba atrybut `DxBind` uzupełnić o własności `Name` i/lub `EventName`, np.

```
[DxBind(Name="Report"]
MyCustomReportName_OnDataSourceDemanded() { /* tu kod bez zmian */ }

[DxBind(Name="Report", EventName="OnDataSourceDemanded")]
CustomMethodName() { /* tu kod bez zmian */ }
```

Jeśli kod referuje do kontrolek raportu (komórki, pola wyliczane, band-y i in.) to te elementy muszą być zadeklarowane (inaczej kod się nie skompiluje). Deklaracje też trzeba opatrzyć atrybutem `DxBind`:

```
[DxBind]
private readonly DevExpress.XtraReports.UI.XRPanel panel;
[DxBind]
private readonly DevExpress.XtraReports.UI.XRLabel label;
[DxBind]
private readonly DevExpress.XtraReports.UI.XRTable table;
[DxBind]
private readonly Soneta.Business.UI.DxReports.Controls.Header header;
[DxBind]
private readonly Soneta.Business.UI.DxReports.Controls.Header footer;
[DxBind]
private readonly DevExpress.XtraReports.UI.DetailReportBand list;
[DxBind]
private readonly DevExpress.XtraReports.UI.DetailReportBand detail;
/* ... */
```

W liście deklaracji muszę się znaleźć tylko te elementy raportu, do których w kodzie metod są referencje.

Należy zauważyć, że z punktu widzenia kompilatora C# powyższe pola nigdy nie są podstawiane wartością, tj. zawsze mają wartość domyślną `null`. Kompilator C# może zgłaszać stosowne ostrzeżenia. Kompilator nie wie jednak, że pola te będą podstawione wartościami, jednak nie wprost, a przez refleksję. Atrybut `DxBind` spowoduje, że przed wykonywaniem metod *event*-ów wartości powyższych pól zostaną podstawione utworzonymi już elementami raportu. Te same elementy są wymienione w pliku `.repx` i trzeba zachować spójność nazw pomiędzy raportem w XML i kodem wydruku.

Osadzenie wydruku : Istnieją dwa sposoby powiązania wydruku z enova:

- wydruk rejestrowany w *assembly*
- wydruk zapisany w bazie

Można przyjąć model miesznay, w którym wydruku `.repx` zapisany jest w bazie, natomiast powiązany kod wydruku wciąż jest pobierany ze skompilowanego *assembly*, więc nie wymaga kompilacji dynamicznej. Jest to szczególnie przydatne wtedy, gdy chcemy zmienić standardowy wydruk enova365, ale tylko w warstwie wizualnej.

Wydruk rejestrowany : Jeżeli chcemy zarejestrować wydruk w *assembly*, traktujemy plik `.repx.cs` jako zwykły plik do kompilacji, razem z innymi źródłami biblioteki. Jeśli `.repx.cs` posiada odwołania do API zewnętrznych narzędzi (DevExpress), potrzebne mogą być osobne licencje na te narzędzia. Dodatkowo konieczne jest, aby zawartość obu plików `.repx` i `.repx.cs` znalazła się w zasobach wynikowego *assembly* (tzw. *Embedded resource*):

- zawartość `.repx` musi się znaleźć w zasobach *assembly*, żeby możliwe było wykonanie wydruku
- zawartość `.repx.cs` musi się znaleźć w zasobach *assembly*, żeby możliwe było odczytanie kodu źródłowego i ew. przeniesienie go do algorytmu kompilowanego dynamicznie

Aby wydruk pojawiał się na określonym widoku w enova365, należy go zarejestrować. Służy do tego atrybut `DxReport`:

```
[assembly: DxReport(
    typeof(Soneta.MyModule.Obiekty), // typ obiektów, których wydruk dotyczy
    "Cennik",                        // nazwa
    "Cennik.repx",                   // powiązany plik .repx
    "CennikTowarow",                 // nazwa view
    Priority = 7)]                   // kolejność w menu
```

Rejestrowanie wydruków jest zalecaną metodą powiązania wydruku z enova. Uzyskujemy w ten sposób wydruk osadzony w enova365 w ten sam sposób, co standardowe wydruki dostarczane z systemem. Uruchomienie wydruku z kodem skompilowanym w DLL jest szybsze, niż wydruku z kodem w formie algorytmu w bazie, bo w całym procesie nie jest potrzeba dynamiczna kompilacja kodu.

Wydruk w bazie : Zarówno plik `.repx` z wizualną warstwą raportu, jak i `.repx.cs` z kodem, mogą być przechowywane w bazie danych.

Wydruk `.repx` zapisujemy w bazie z menu *Ustawienia wydruków* na wybranej liście lub formularzu. Otwarcie wydruku do edycji powodje skopiowanie go do bazy. Kod wydruku możemy dodać do konfiguracji enova365 w *Ustawienia -> Raportowanie -> REPX -> Snippet-y*.

Wydruki przechowywane w bazie nie wymagają dodatku DLL, stąd łatwiej je wdrożyć. Jeśli jednak wraz z wydrukiem przeniesiemy do bazy danych jego kod, uruchamianie wydruku będzie wiązało się jednak z dodatkowym kosztem, jakim jest dynamiczna kompilacja kodu wydruku. Po przeniesieniu do bazy *snippet* staje się algorytmem kompilowanym dynamicznie, analogicznie do tasków, tupli, cech wyliczanych i wielu innych. Domyślnie algorytmy *snippet*-ów dodawane są do projektu `Soneta.Runtime.Reports`. Algorytmy kompilowane są w ramach swoich projektów. Oznacza to, że przy pierwszym wykonywanym wydruku z algorytmem skompilowane zostaną wszystkie inne algorytmy z tego samego projektu.

Jeśli skopiujemy do bazy danych raport `.repx`, ale pozostawimy *snippet* bez zmian (nie przeniesiemy go do bazy), kod skryptu również nie będzie się kompilował dynamicznie.

Powiązanie wydruku z kodem : Mając wzorzec, albo zarejestrowany w *assembly*, albo zapisany w bazie, możemy powiązać wydruk `.repx` z jego snippet-em `.repx.cs`. W obu wariantach powiązanie *snippet*-a z raportem odbywa się podobnie. Otwieramy w tym celu raport do edycji, wybieramy *Komponenty -> Dodaj źródło danych -> Dodaj kodowane źródło danych*, i dodajemy do raportu dodatkowy komponent, zgodnie z tutorialem.

Do *snippet*-a możemy przypisać albo **Typ obiektu** - dla tych pochodzących z *assembly*, albo **Plik do kompilacji** - dla tych pochodzących z algorytmów w bazie. **Typ obiektu** będzie dostępny na liście po przekompilowaniu dodatku. Na liście dostępne są wszystkie publiczne klasy z *assembly*, dziedziczące z klasy `ReportSnippet`. W liście wyboru **Plik do kompilacji** dostępne są snippet-y dodane w *Ustawienia -> Raportowanie -> REPX -> Snippet-y*.

Snippet jako algorytm : Jeśli mamy standardowy wydruk, który chcemy otworzyć do edycji, wymaga to przeniesienia wydruku zarejestrowanego w *assembly* do bazy danych. Podczas otwarcia standardowego wydruku do edycji domyślnie przenoszony jest do bazy tylko `.repx` czyli *layout*, wizualna warstwa raportu. Natomiast kod logiki wydruku, czyli *snippet*, pozostaje bez zmian. Jeśli kod wydruku także wymaga modyfikacji, możemy przenieść standardowy *snippet* dostarczony z *assembly* do bazy danych. Służy do tego worker **Utwórz ze standardowego** dostępny w *Ustawienia -> Raportowanie -> REPX -> Snippet-y*. Worker wymaga jedynie podania typu standardowego *snippet*-a oraz docelowej nazwy. Lista dostępnych typów to ta sama lista, która dostępna jest przy podpinaniu *snippet*-a do raportu.

Konwersja automatyczna

Konwersję wydruków z postaci C# do XML trudno wykonać całkowicie automatycznie, jeśli raport zawiera skrypty. Konwersja wymaga edycji kodu wydruku. Jednak jak pokazano w poprzednich sekcjach, jest to edycja powierzchowna, wymagająca praktycznie dodania atrybutów, bez zmian definicji metod.

Automatyczną konwersję większych zestawów wzorców mogą wspomóc narzędzia dostarczane przez DevExpress i Soneta.

Aby utworzyć instancję raportu na potrzeby konwersji można skorzystać ze standardowej metody

```
var report = (XtraReport)System.Activator.CreateInstance(<typ raportu>);
```

Można także utworzyć instancję obiektu `XtraReport` i zasilić ją wzorcem wczytanym z pliku `.repx` C#:

```
var report = new XtraReport();
```

```
report.LoadLayout(...);
```

Metoda `XtraReport.LoadLayout()`, wczytuje wydruk ze starszego formatu `repx`. Podobną operację wykonuje statyczna metoda `XtraReport.FromStream`.

Wzorzec taki można także zapisać przy pomocy metody `XtraReport.SaveLayout()`:

```
XtraReport report;
...
using (var ms = new MemoryStream())
    report.SaveLayout(ms); // tylko .NET Framework
```

Odczyt i zapis wzorców w postaci XML realizują analogiczne metody `XtraReport.SaveLayoutToXml()` oraz `XtraReport.LoadLayoutFromXml()`. Działają analogicznie jak opisane wcześniej metody `SaveLayout()` i `LoadLayout()`, ale akceptują zawartość XML zamiast C#. Utworzenie instancji wydruku z wzorca XML realizuje także statyczna metoda `XtraReport.FromXmlStream`.

Dodatkowo, wewnętrzne API enova365 udostępnia pomocniczą klasę `Soneta.Business.UI.DxReports.DxReportLoader` oraz statyczne metody obsługujące oba formaty C# i XML:

```
public static XtraReport DxReportLoader.Load(Stream stream);
public static XtraReport DxReportLoader.Load(string file);
```

Druga forma metody `Load` obsługuje raporty osadzone w bazie, zatem `string file` może być nazwą raportu osadzonego w *assembly* jako `Embedded resource`.

W metodzie `Load()` użyta jest wykorzystywana już wcześniej publiczna klasa `DxReportConverterFromStandard`, odpowiedzialna za automatyczne rozpoznanie różnic między formatami `repx` i `repx.cs`:

```
var converter = new DxReportConverterFromStandard { InputStream = ... };
converter.Convert();
return converter.Report;
```

Zapisując wzorzec do postaci XML należy sprawdzić, czy w gdzieś XML-u nie pozostają ścieżki to lokalnych zasobów użytkownika, np. styli lub obrazków. We wzorcach XML nie powinny znaleźć się ścieżki do zasobów, bo są to ścieżki adekwatne dla komputera programisty, a nie wykonującego wydruk.

Jeżeli wzorzec zawiera skrypty, instancja wydruku będzie posiadać niepuste *property* `ScriptsSource`:

```
XtraReport report;
...
var script = report.ScriptsSource;
if (script != null) {
    ... // należy zamienić na snippet
```

```
    report.ScriptsSource = null;
}
```

Wzorzec wydruku w nowej postaci nie powinien posiadać skryptów. Kod skryptu należy zamienić na *snippet*, czyli na klasę dziedziczącą z `Soneta.Business.UI.DxReports.ReportSnippet`:

```
namespace MyReports
{
    public class MyReportSnippet : ReportSnippet
    {
        /* ew. klasa parametrów (bez zmian) */
        public class MyReportParams : ContextBase
        {
            public MyReportParams(Context cx) : base(cx) { ... }
        }

        /* deklaracje elementów do których odwołuje się kod event-ów */
        [DxBind] private DevExpress.XtraReports.UI.DetailBand band;
        [DxBind] private DevExpress.XtraReports.UI.XRLabel label;
        [DxBind] private DevExpress.XtraReports.UI.CalculatedField field;
        ...

        /* definicje event-ów */
        [DxBind]
        private void label_BeforePrint(object sender, CancelEventArgs e) {
            ...
            label.Text = "dynamicznie wyliczana wartość etykiety";
        }

        [DxBind]
        private void field_GetValue(object sender, DevExpress.XtraReports.UI.GetValueEventArgs e)
{
            ...
            e.Value = "wartość pola wyliczanego";
        }
    }
}
```

Zmiany w DevExpress 23 / enova 2406

W wersji enova 2406 biblioteki DevExpress zostały zaktualizowane do wersji 23, czyli wersji zgodnej z platformą .NET 8. DevExpress, podobnie jak cały świat .NET, odchodzi od technologii GDI+, wykorzystywanej w klasach przestrzeni nazw `System.Drawing` (`Font`, `Image`, `PaperKind`, `PrinvEventArgs` i in.). Zgodnie z wcześniejszymi zapowiedziami, powoduje to istotne niezgodności w API dostarczanym przez DevExpress, w szczególności w wydrukach REPX. W konsekwencji, wiele wydruków REPX przechowywanych w bazach danych skonwertowanych do wersji 2406, wymaga zamiany do postaci zgodnej z DevExpress 23 i .NET 8. enova365 dostarcza narzędzie wspomagające konwersję. Jeśli zalecana przy wersji 2304 rewizja wydruków (przejście na snippety) nie została wykonana, stanowczo rekomendujemy wykonanie jej przy okazji podnoszenia instalacji do wersji 2406.

Rewizję i konwersję wydruków należy bezwzględnie wykonać zaraz po podniesieniu bazy danych, przed dostarczeniem nowej wersji klientowi.

Snippety są mechanizmem enova 2304, wprowadzonym z maksymalnym możliwym zachowaniem zgodności wstecz:

- obsługiwane są nadal wydruki w postaci kodu
- obsługiwane są nadal skrypty zaszyte w wydrukach

Natomiast zmiany wprowadzone przez DevExpress 23 (enova 2406) w sposób bezwarunkowy, bez zachowania zgodności wstecz, łamią kontrakt API starszych wydruków. Oznacza to, że wydruki REPX działające do wersji enova 2404 włącznie, wraz z wersją enova 2406 mogą przestać działać, gdyż ich kod przestaje być zgodny z API dostarczanym przez DevExpress. Dotyczy to zarówno kodu w starszych wydrukach jak i kodu w snippetach. Wszystkie wydruki standardowe, tj. dostarczane wraz z systemem enova 2406, zostały odpowiednio dostosowane do nowych technologii. Wydruki własne, czyli przechowywane w bazie danych w dowolnej formie, wymagają modyfikacji. Jeśli w instalacji istnieją wydruki uruchamiane z przestrzeni dyskowej (poza bazą), należy je wprowadzić do bazy i skonwertować.

Trzeba pamiętać, że większość wydruków korzysta z pewnych wspólnych komponentów, takich jak nagłówki, stopki, arkusze styli, i inne podraporty. Jeśli którykolwiek z tych komponentów mieszka w bazie danych, to traktowany jest przez enova jako **niestandardowy**. Oznacza to, że kolejne wersje enova nie mogą aktualizować tych komponentów, i w podnoszonych bazach danych mogą znajdować się ich bardzo stare wersje. W konsekwencji, nawet jeśli dany wydruk nie był modyfikowany w danej instalacji (jest standardowy), to z dużym prawdopodobieństwem korzysta z nagłówków i stopek (podraportów), które są zapisane w bazie. Czyli są traktowane jako niestandardowe, nie są aktualizowane wraz z nowymi wersjami enova, i wymagają modyfikacji.

W kolejnych sekcjach opisane zostały najistotniejsze zmiany w API dostarczanym przez DevExpress. Wymienione narzędzie wspomagające konwersję obsługuje opisane niżej niezgodności, tzn. stara się automatycznie podmienić niekompatybilne konstrukcje programistyczne na zamienniki zgodne z API. Ponieważ pełen semantyczny kontekst kodu zna tylko programista, rezultat z konwertera automatycznego wymaga weryfikacji.

Zamiana PrintEventArgs na CancelEventArgs

Najistotniejszą ilościowo zmianą w API DevExpress jest podmiana typu `System.Drawing.Printing.PrintEventArgs` na `System.ComponentModel.CancelEventArgs` we wszystkich sygnaturach metod eventów.

```
void XtraReport_BeforePrint(object sender, PrintEventArgs e) // błąd, sygnatura metody nie pasuje do eventu
void XtraReport_BeforePrint(object sender, CancelEventArgs e) // ok
```

Ponieważ zmiana dotyczy sygnatur metod wykorzystywanych w powszechnych eventach takich jak `BeforePrint`, ta niezgodność dotyczy prawdopodobnie niemal wszystkich wydruków i snippetów.

Zamiana Font na DXFont

Wiele powszechnych kontrolek raportu (`cell`, `table`, `band`, `report`, ...) posiada property `Font`. Dawniej typem tego property był `System.Drawing.Font`, teraz jest to `DevExpress.Drawing.DXFont`.

```
cell.Font = new System.Drawing.Font("Calibri", 9.0f); // niezgodny typ
cell.Font = new DevExpress.Drawing.DXFont("Calibri", 9.0f); // ok
```

DevExpress wprowadza swoje zamienniki dla typów z przestrzeni `System.Drawing`, w tym dla fontów. Zamienniki te posiadają zaimplementowane operatory rzutowania (*implicit cast operator*), przez co próba podstawienia obiektu `System.Drawing.Font` do property typu `DevExpress.Drawing.DXFont` nie zakończy się błędem. Spowoduje natomiast ostrzeżenie, że jest to operacja dostępna **tylko dla Windows** (utworzenie instancji fontu wymaga użycia natywnie windowsowej technologii GDI+).

Brak odpowiednika FontFamily

DevExpress nie wprowadza zamiennika dla typu `System.Drawing.FontFamily`. Jedyną reprezentacją rodziny fontów jest jej nazwa.

```
cell.Font = new System.Drawing.Font(new System.Drawing.FontFamily("Calibri"), 9.0f); // jw.,
niezgodny typ dla podstawienia
cell.Font = new DevExpress.Drawing.DXFont("Calibri", 9.0f); // ok
```

Zamiana enum FontStyle na DXFontStyle

Wszędzie, gdzie używany był wcześniej enum `System.Drawing.FontStyle` (bold, italic, ...) powinien być teraz użyty `DevExpress.Drawing.DXFontStyle`. Dla enum nie da się w C# zrobić *implicit cast operator*, nie da się tym samym zrobić automatycznej konwersji, jaką zastosowano dla `Font` i `DXFont`. Oznacza to, że użycie `System.Drawing.FontStyle` będzie powodować błąd kompilacji.

Zamiana Image na DXImage

Użycia `System.Drawing.Image` należy zamienić na `DevExpress.Drawing.DXImage`. Property `Image` w kontrolkach DevExpress jest *obsolete*, należy używać ImageSource. W wydrukach enova nie występuje obiekt typu `Image`, bo do wyświetlania obrazków na wydrukach używana jest własna kontrolka `ResourcePictureBox`.

Zamiana PaperKind na DXPaperKind

Użycia enum `System.Drawing.Printing.PaperKind` należy zamienić na `DevExpress.Drawing.Printing.DXPaperKind`.

Zamiana GraphicsUnit na DXGraphicsUnit

Użycia enum `System.Drawing.GraphicsUnit` należy zamienić na `DevExpress.Drawing.DXGraphicsUnit`.

Zamiana StringTrimming na DXStringTrimming

Użycia enum `System.Drawing.StringTrimming` należy zamienić na `DevExpress.Drawing.DXStringTrimming`.

Zamiana Margins na DXMargins

Użycia typu `System.Drawing.Printing.Margins` należy zamienić na `DevExpress.Drawing.Printing.DXMargins`. Propertiesy obiektu `System.Drawing.Printing.Margins` (`Left`, `Right`, `Top`, `Bottom`) są typu `int`. Propertiesy obiektu `DevExpress.Drawing.Printing.DXMargins` są typu `float`. Konstruktor `DXMargins` akceptuje parametry `int`. Natomiast ew. operacje arytmetyczne na marginesach mogą się nie kompilować i wymagają korekty:

```
var width = 2100 - (Margins.Right + Margins.Left); // błąd, niezgodność typów int i float
var width = 2100 - (int)(Margins.Right + Margins.Left); // ok
```

Inne materiały

Tematyka wydruków REPX była szeroko omawiana na konferencjach LabIT i GeekOut w latach 2023 i 2024.

Tutaj nagranie wykładu o przyczynach zmian w repx od wersji 2304, genezie snippet-ów, customizacji wydruków i in.

Przykład dodatku pokazującego worker uruchamiający wydruki w kilku wariantach, a także pokazujący sposób osadzenia wydruku w dodatku znajduje się w repozytorium na github

Dokumentacja workera wspomagającego konwersję wydruków do postaci zgodnej z DevExpress 23 i .NET 8 w osobnym artykule.

Soneta sp. z o.o.
ul. Wadowicka 8A, 30-415 Kraków
+48 12 349 28 00
kontakt@enova.pl